

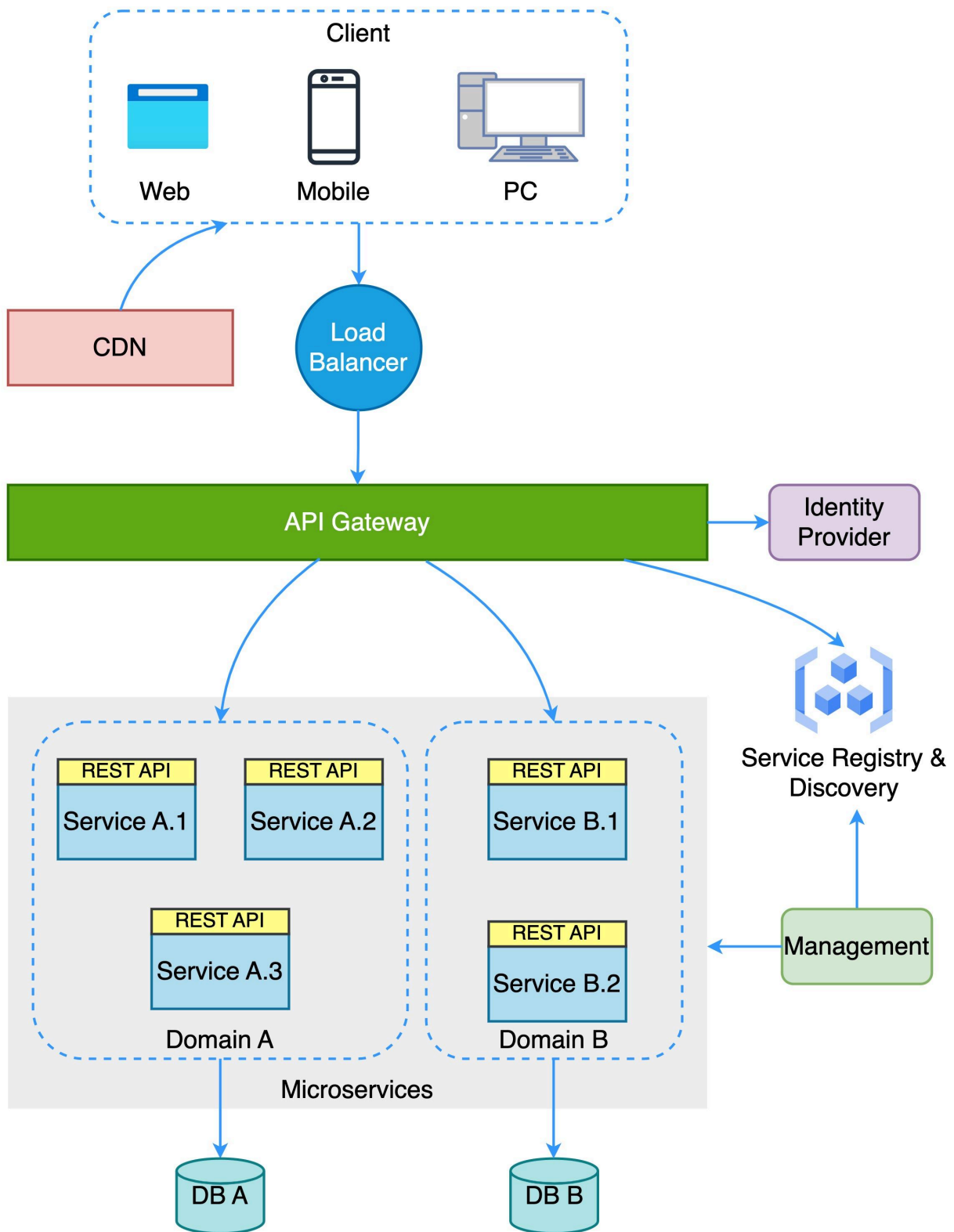
SA diagrams

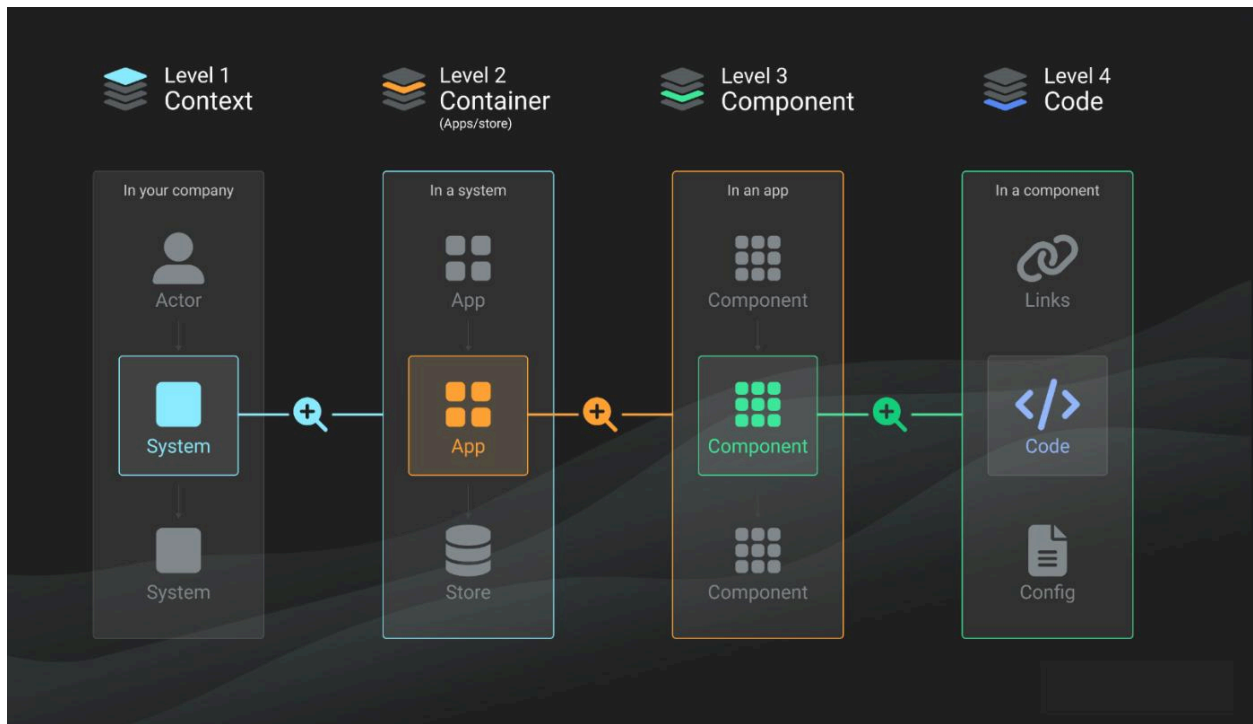
If you're a **Software Architect**, you don't need to memorize every diagram notation, but you should be comfortable choosing the right diagram for the right architectural question.

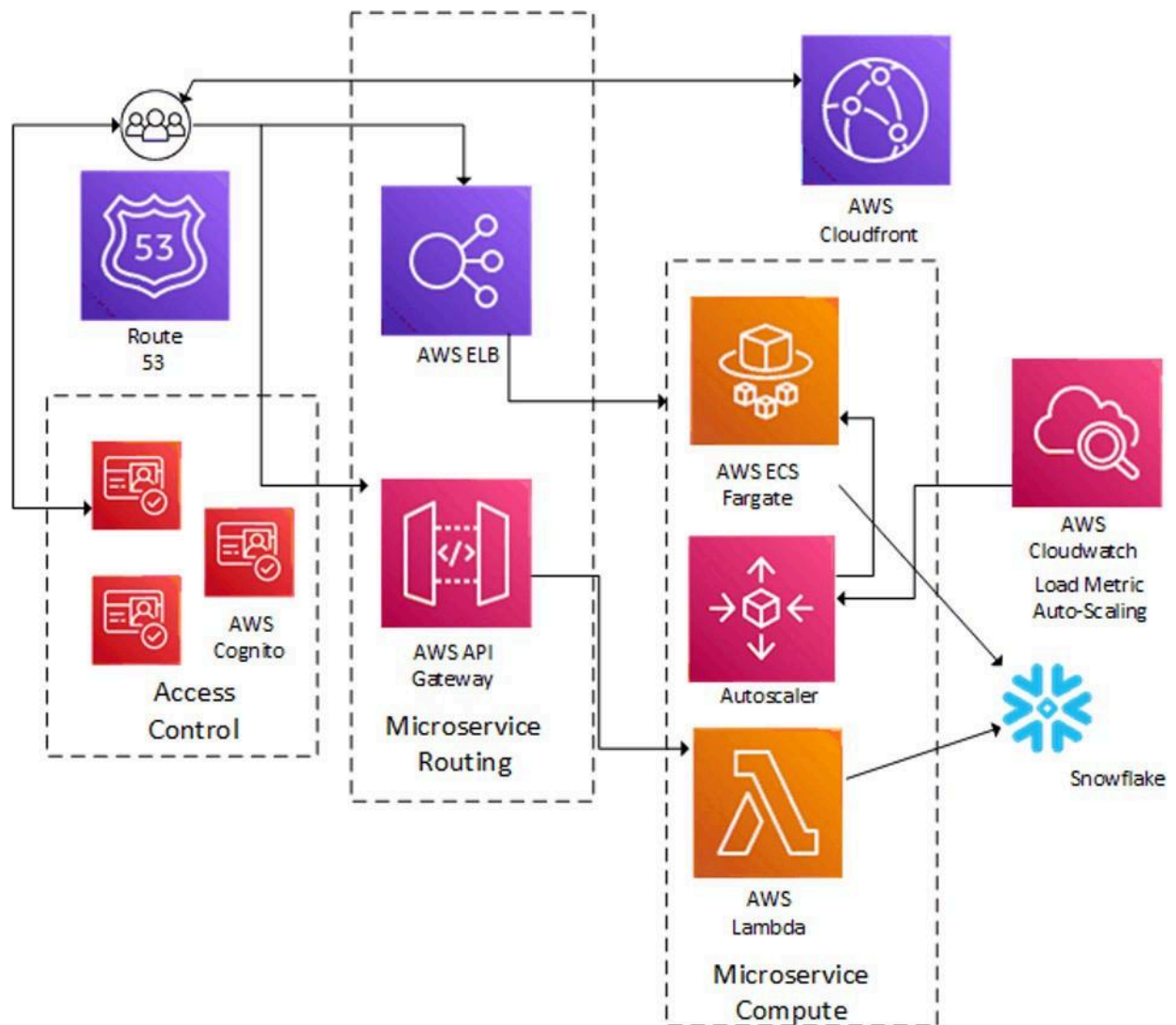
Here is a practical **Software Architect diagram toolbox**, from most important to more specialized.

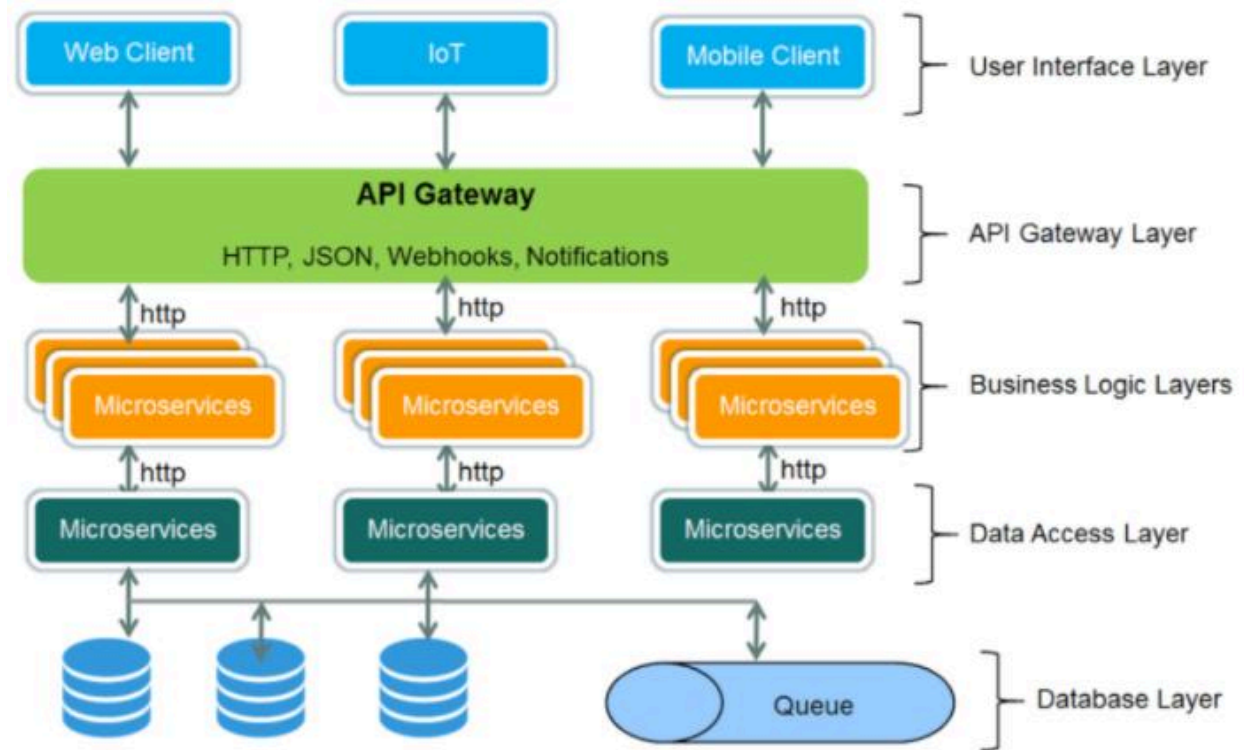
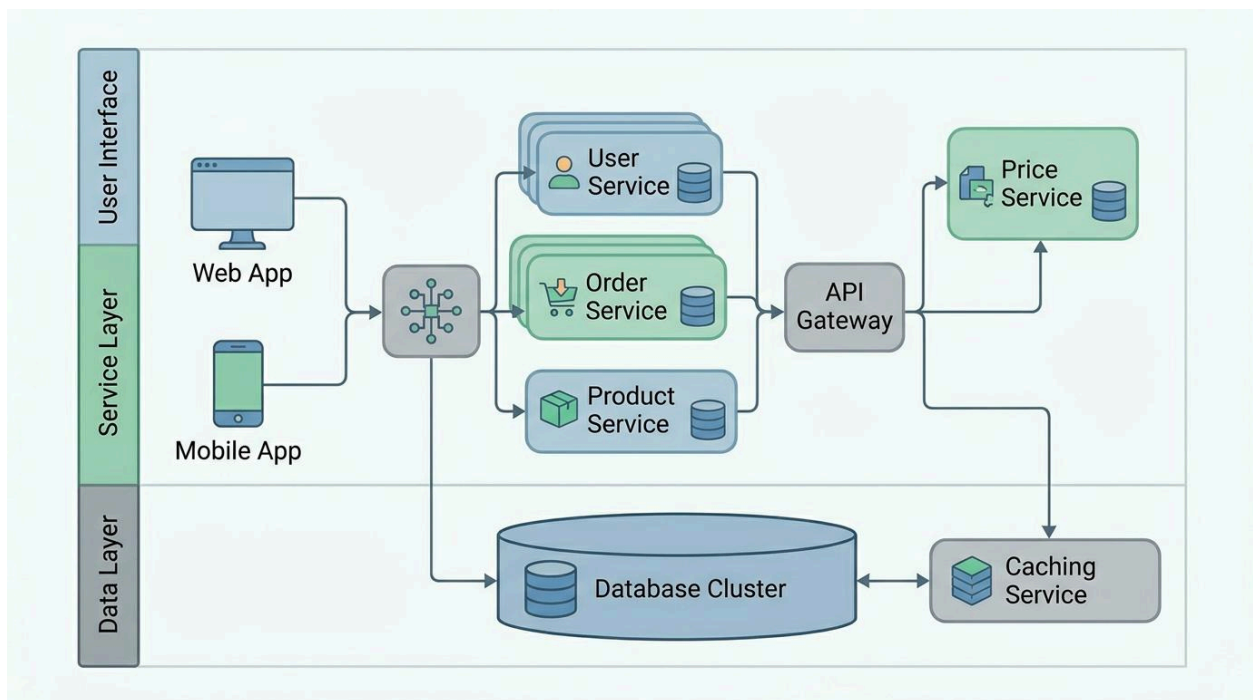
1. Architecture & System-Level Diagrams ★★ ★

Microservice Architecture

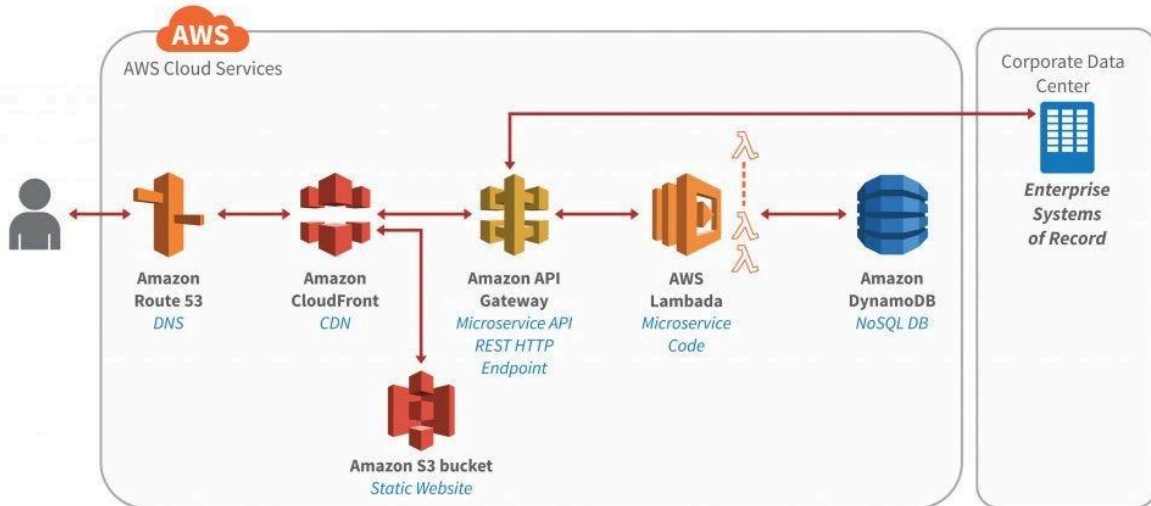








Cloud Serverless Microservice Architecture



Courtesy: Amazon Web Services

Diagram	What it answers
System Context Diagram	Who/what interacts with our system?
Container Diagram	What are the major applications/services/data stores?
Component Diagram	What components exist inside a service/application?
Deployment Diagram	Where does everything run?
Solution Architecture Diagram	How does the whole solution fit together?
Logical Architecture Diagram	What are the major logical building blocks?
Physical Architecture Diagram	What infrastructure actually hosts them?
Cloud Architecture Diagram	How do AWS/Azure/GCP resources fit together?
Reference Architecture	What architecture pattern should teams follow?
Landscape Diagram	What systems exist across the organization?

Especially important: C4 Model

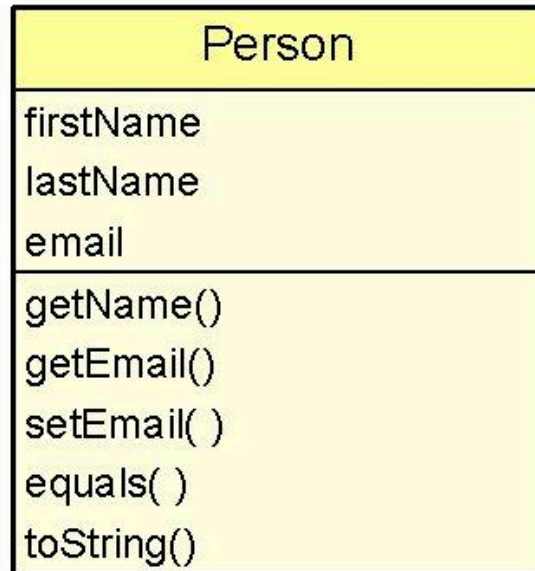
As an architect, I would prioritize:

Context → Container → Component → Code

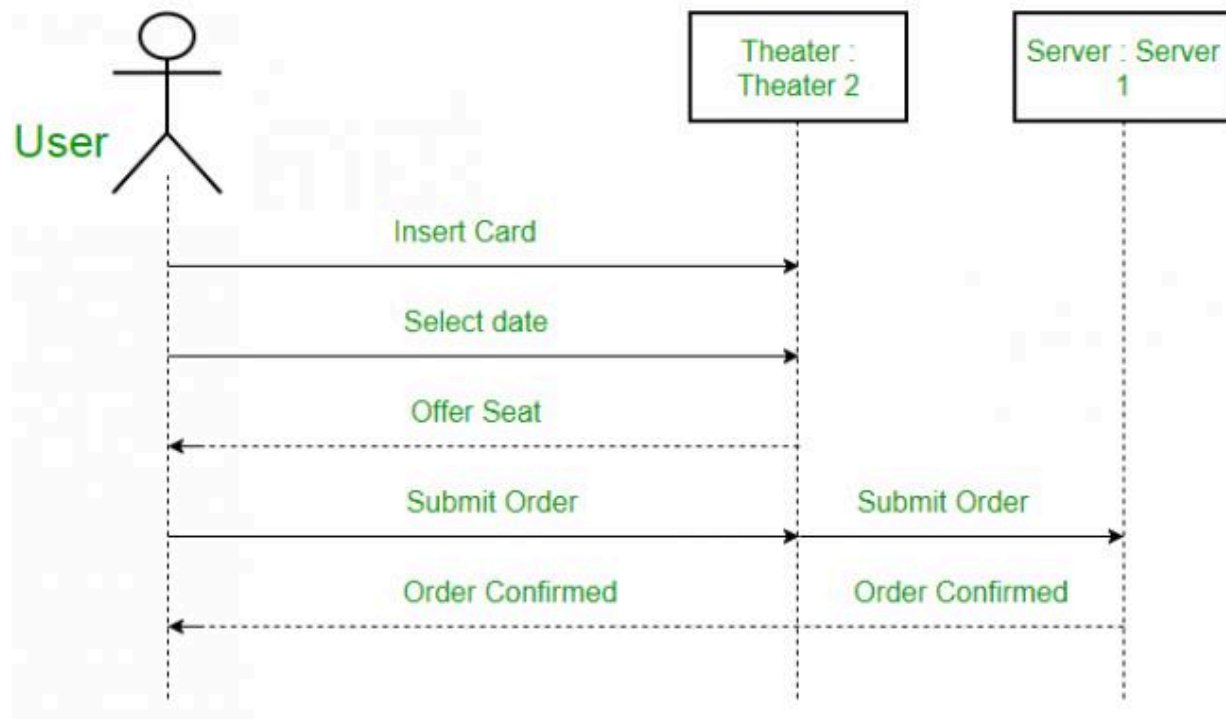
You generally don't need to draw the Code level unless developers need it.

2. UML Diagrams ★★

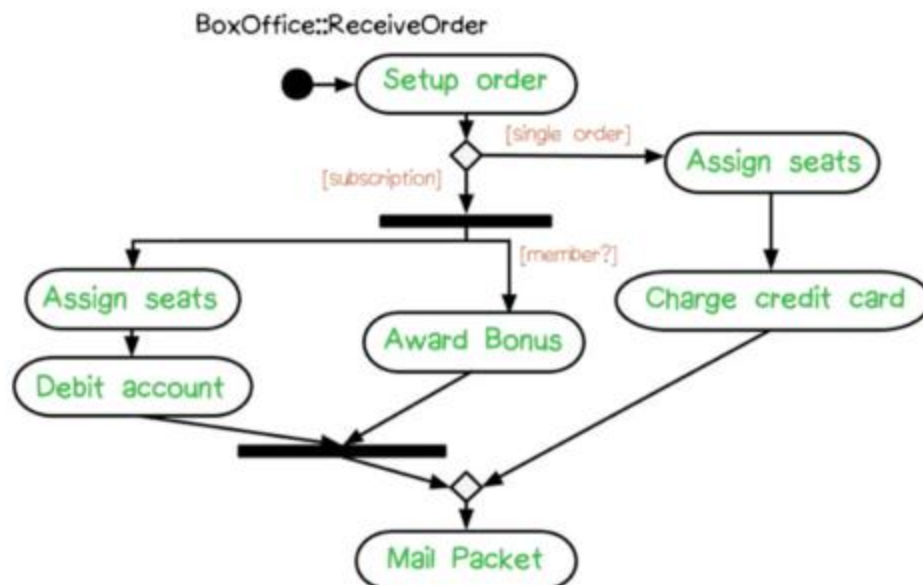
Example: UML Class Diagram



1-5



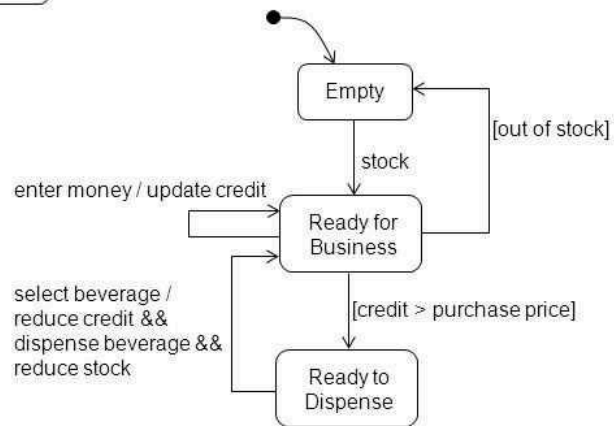
Example Activity Diagram



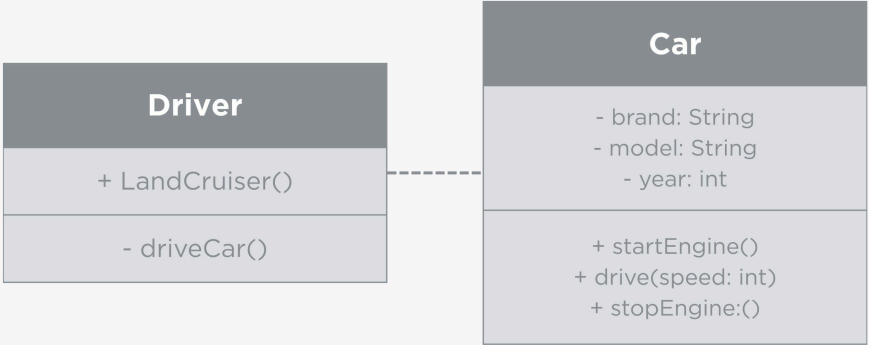
UML State Machine Diagram

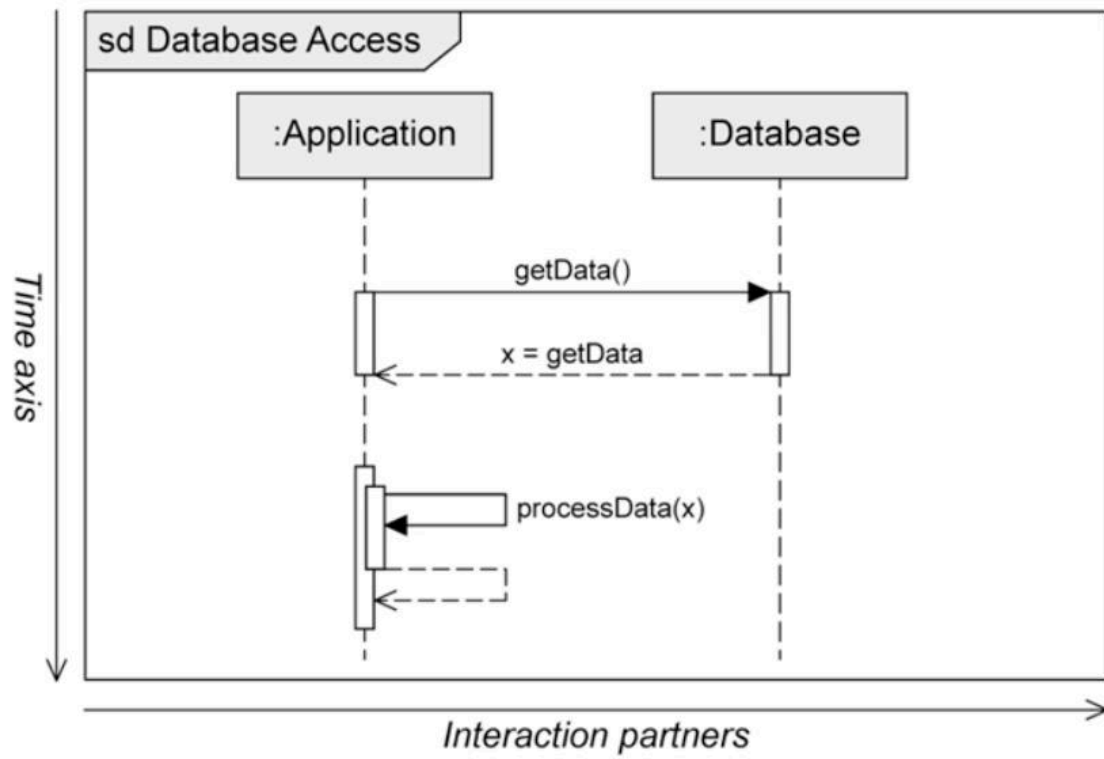


Generic Form

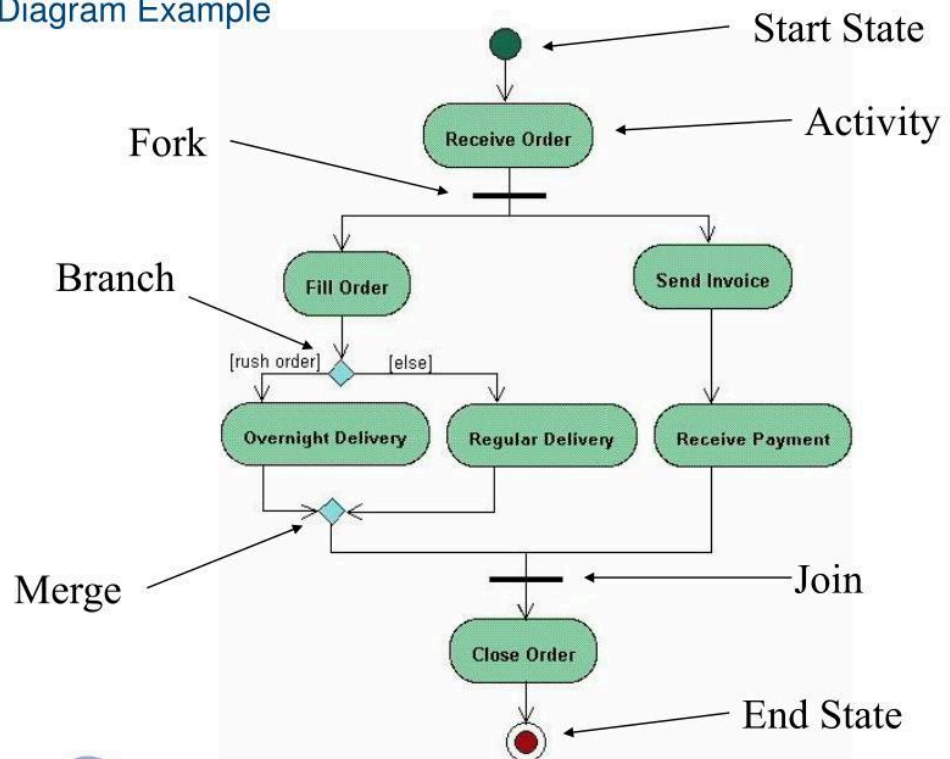


Example





Activity Diagram Example



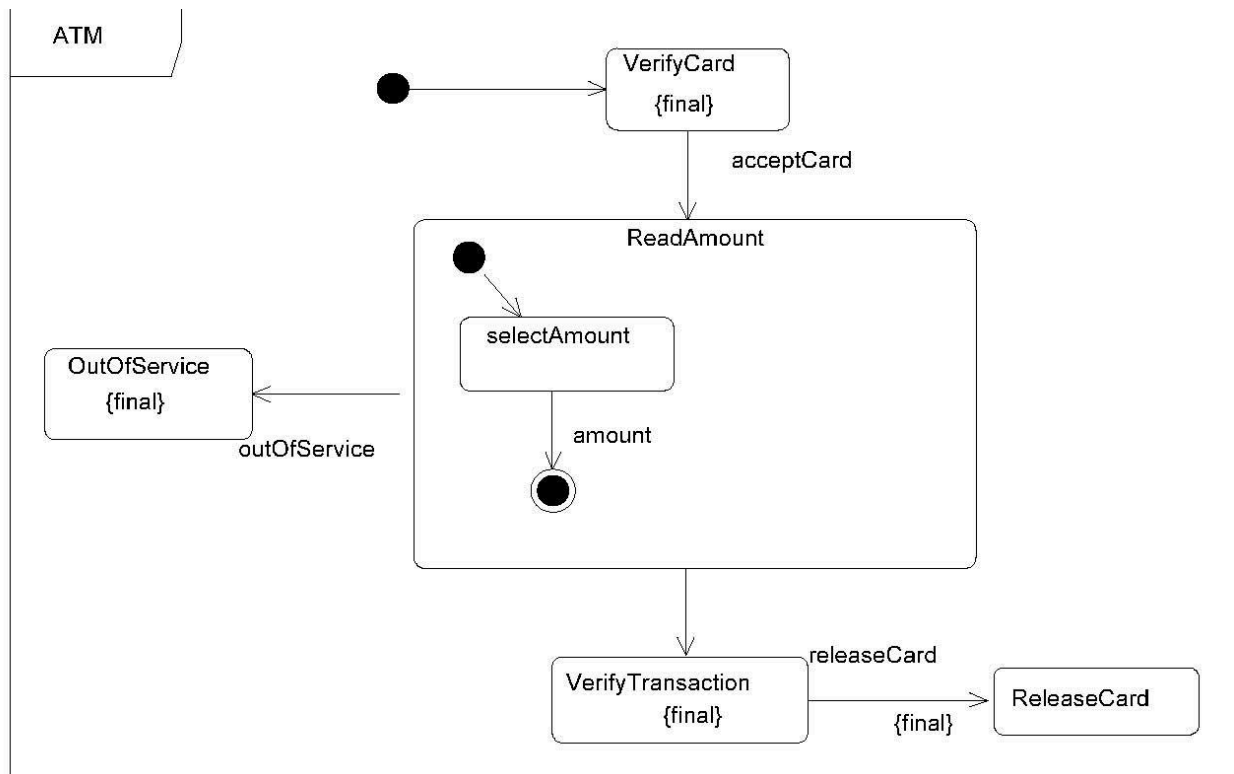


Figure 15.42 - A general state machine

Structural UML

1. Class Diagram
2. Component Diagram
3. Object Diagram
4. Package Diagram
5. Composite Structure Diagram
6. Deployment Diagram
7. Profile Diagram

Behavioral UML

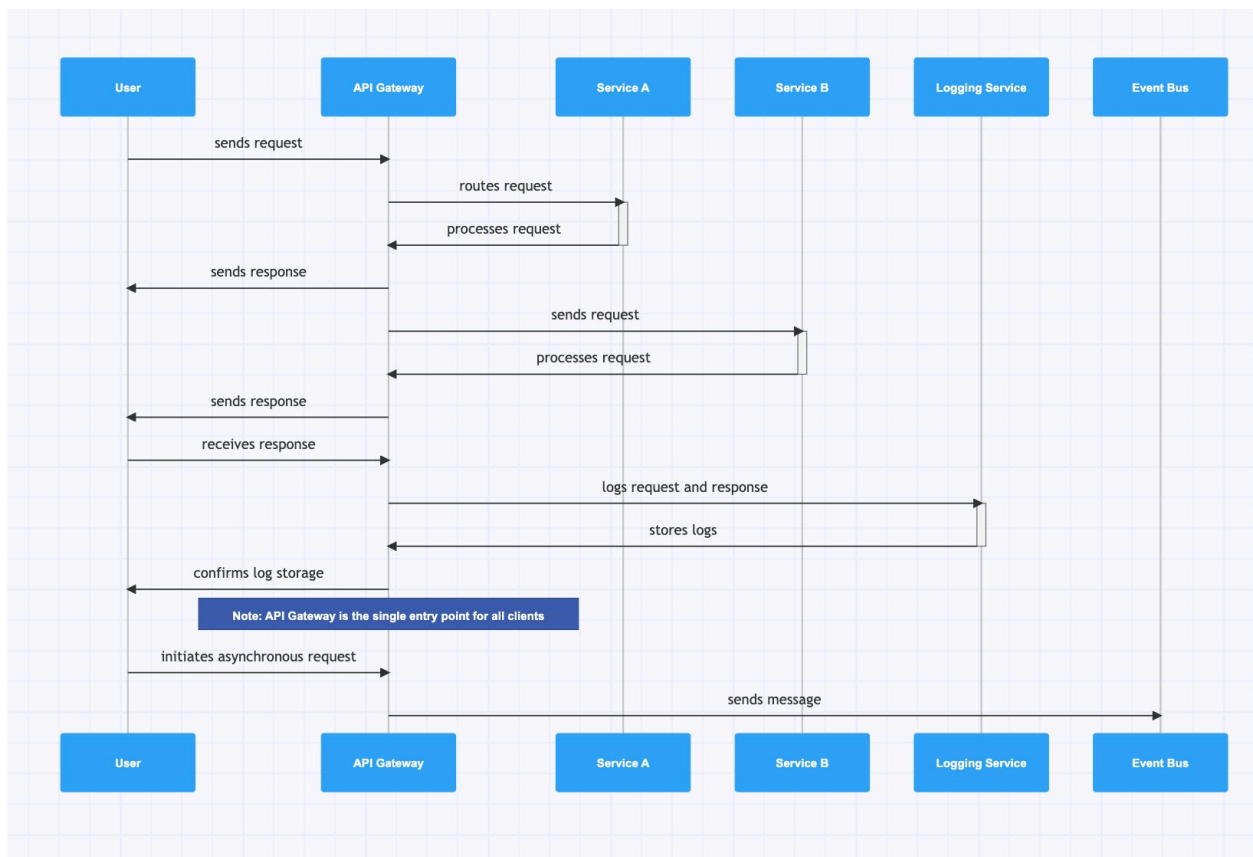
1. Sequence Diagram ★
2. Activity Diagram
3. State Machine Diagram ★

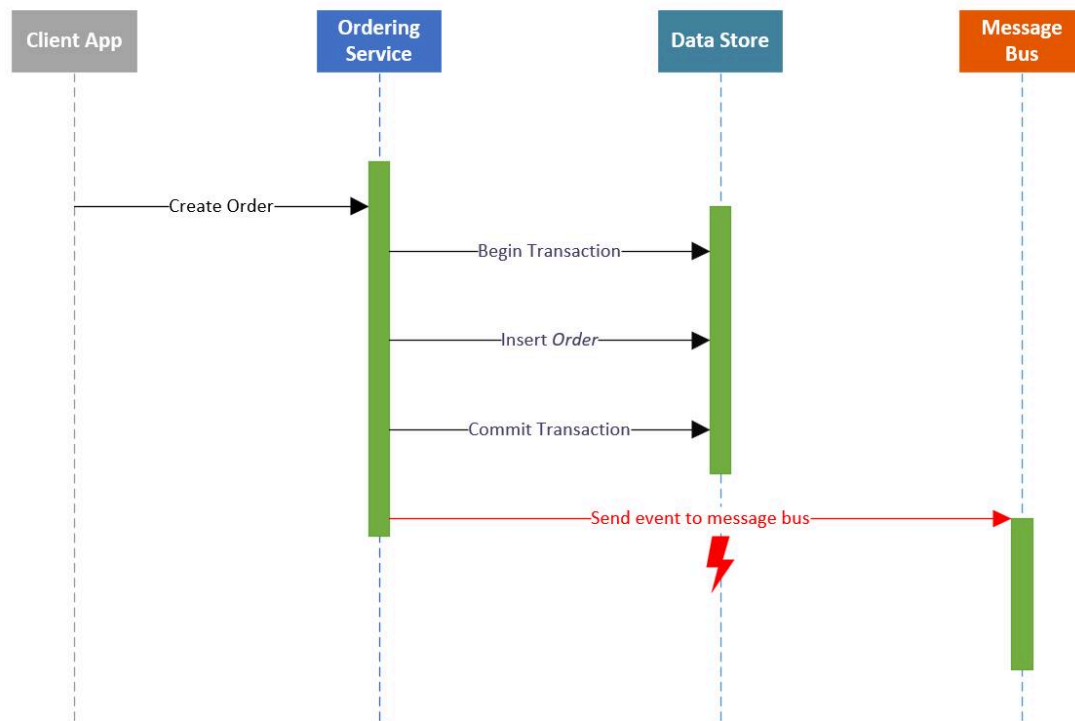
4. Use Case Diagram
5. Communication Diagram
6. Interaction Overview Diagram
7. Timing Diagram

For architecture work, the most useful are:

Class + Component + Sequence + State Machine + Deployment

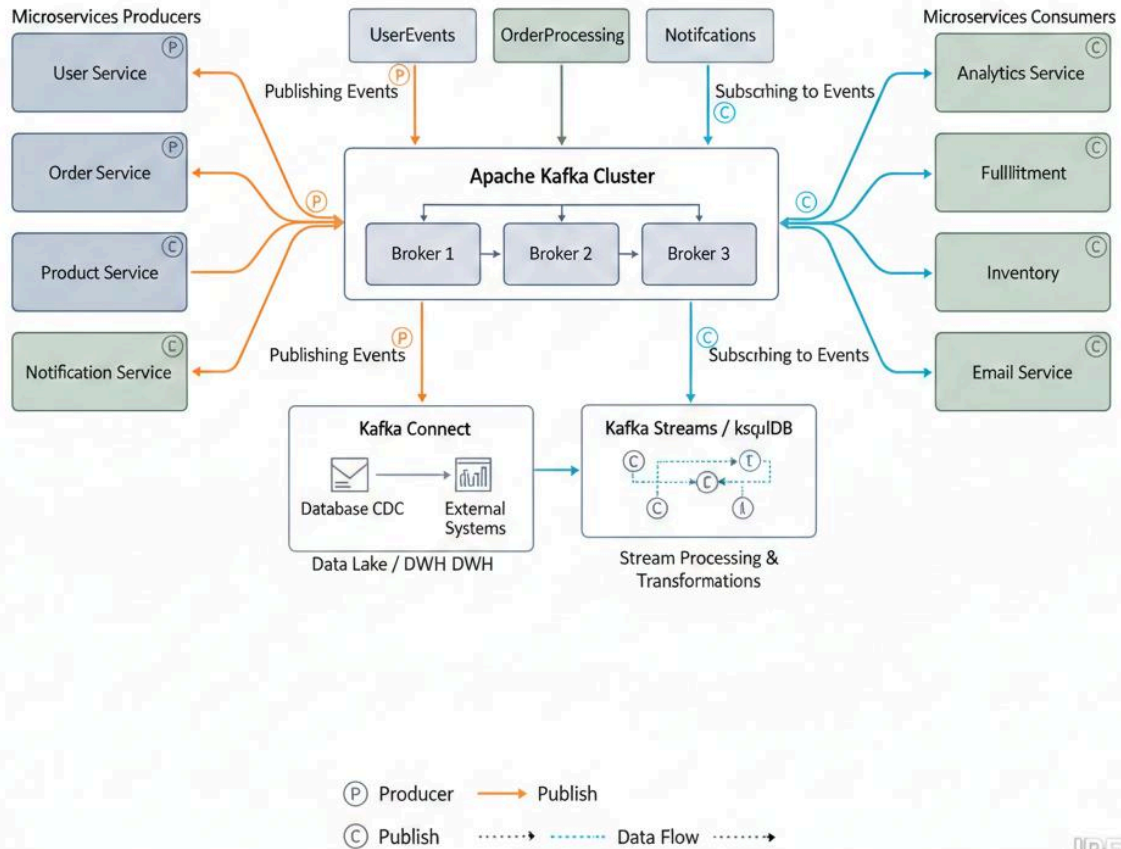
3. Interaction & Runtime Diagrams ★★

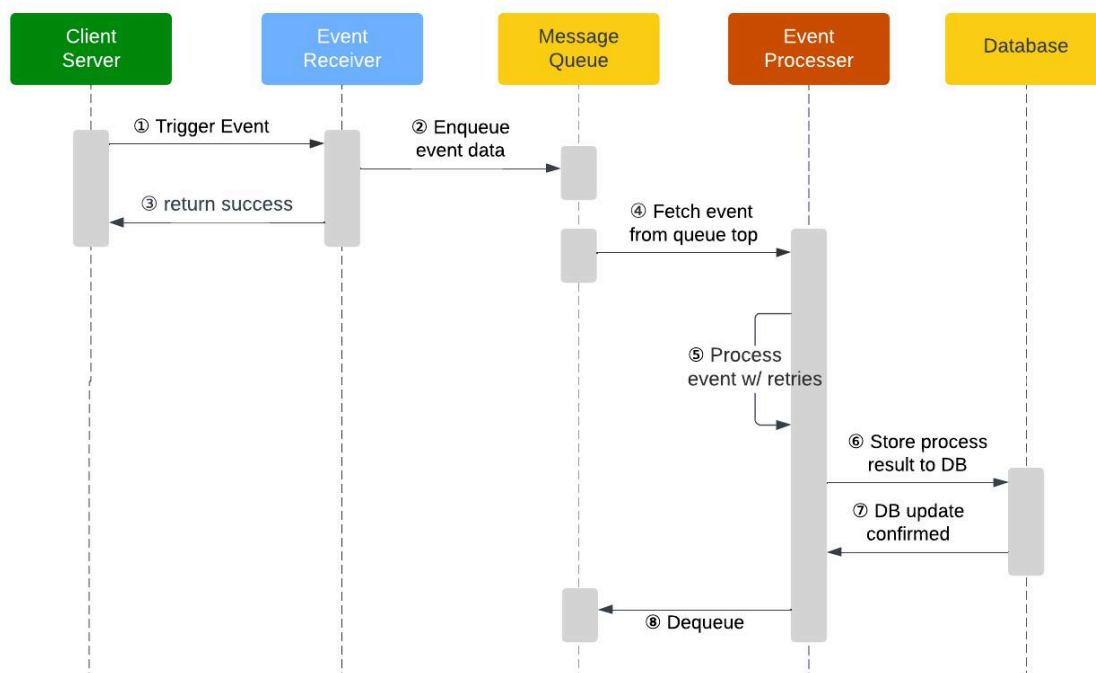
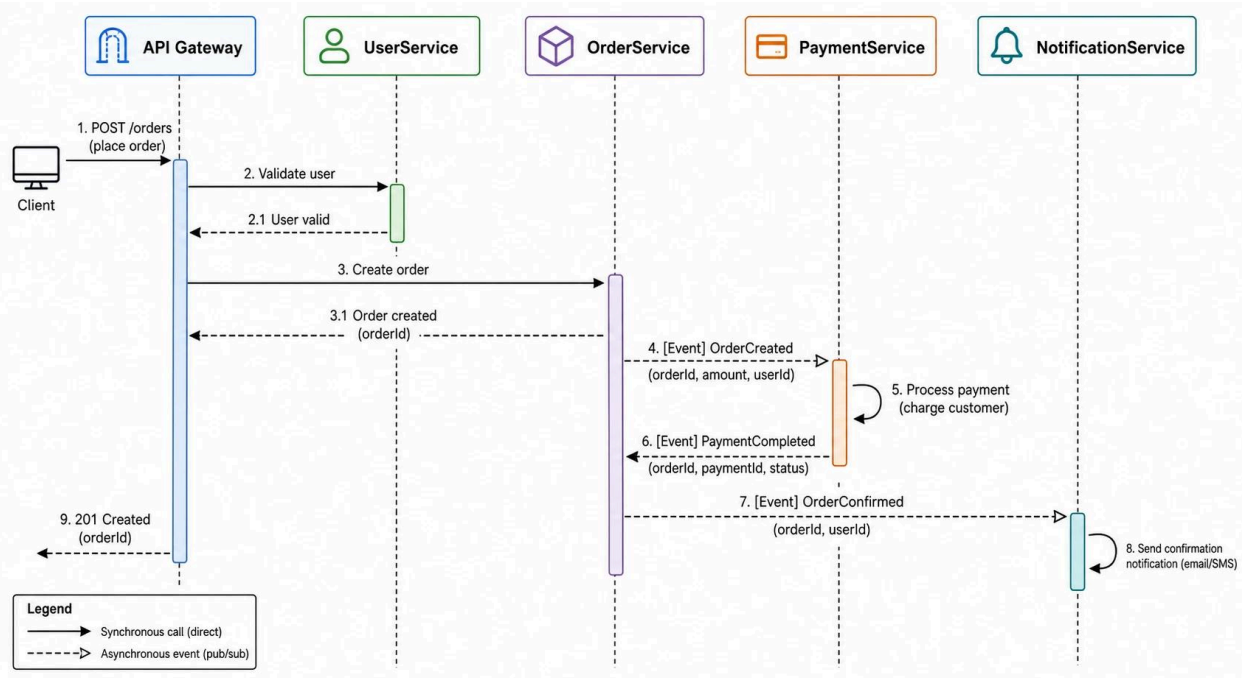


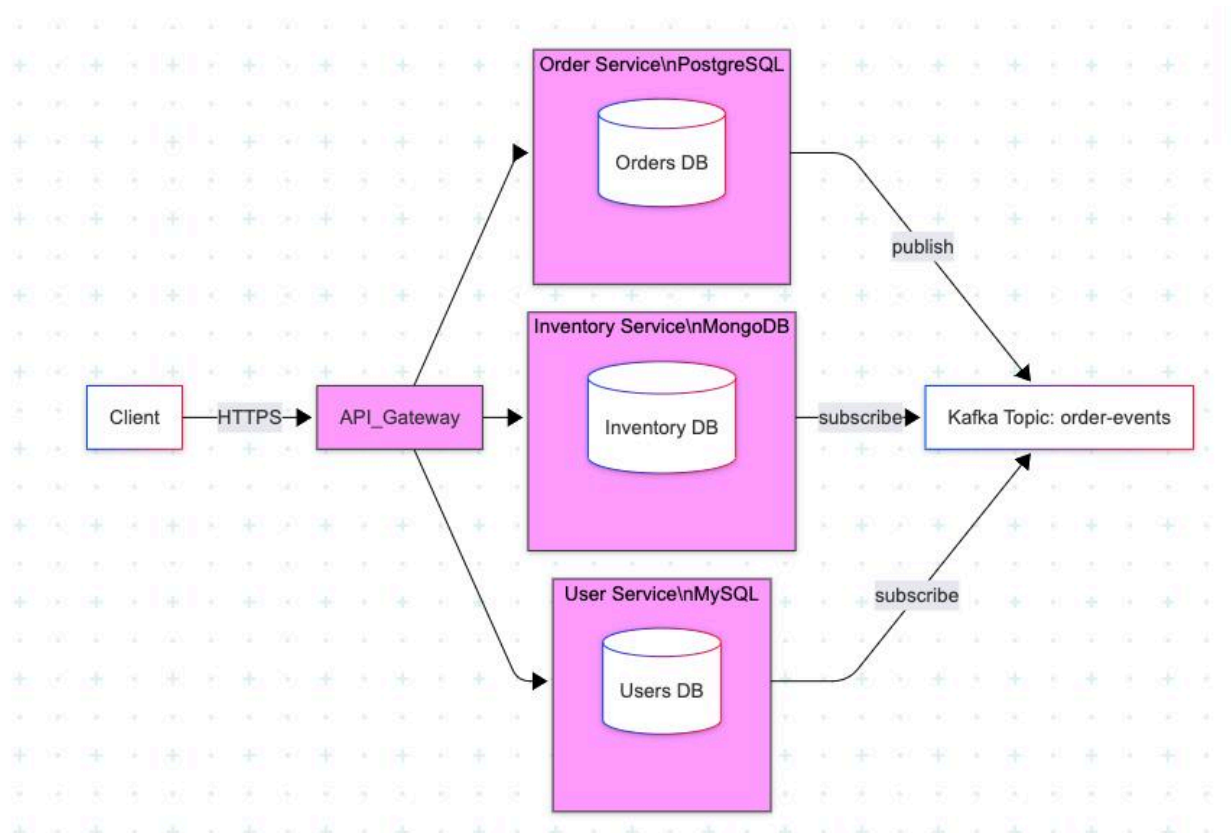


Microservices Architecture with Apache Kafka

Event-Driven Data Flow and Integration







These are extremely important for distributed systems.

Sequence Diagram

Shows:

```

sequenceDiagram
    participant Client
    participant API
    participant Order_Service as Order Service
    participant Payment_Service as Payment Service
    participant Kafka
    participant Notification_Service as Notification Service

    Client->>API: 
    activate API
    API->>Order_Service: 
    deactivate API
    activate Order_Service
    Order_Service->>Payment_Service: 
    deactivate Order_Service
    activate Payment_Service
    Payment_Service->>Kafka: 
    deactivate Payment_Service
    activate Kafka
    Kafka->>Notification_Service: 
    deactivate Kafka
    activate Notification_Service
  
```

Useful for understanding:

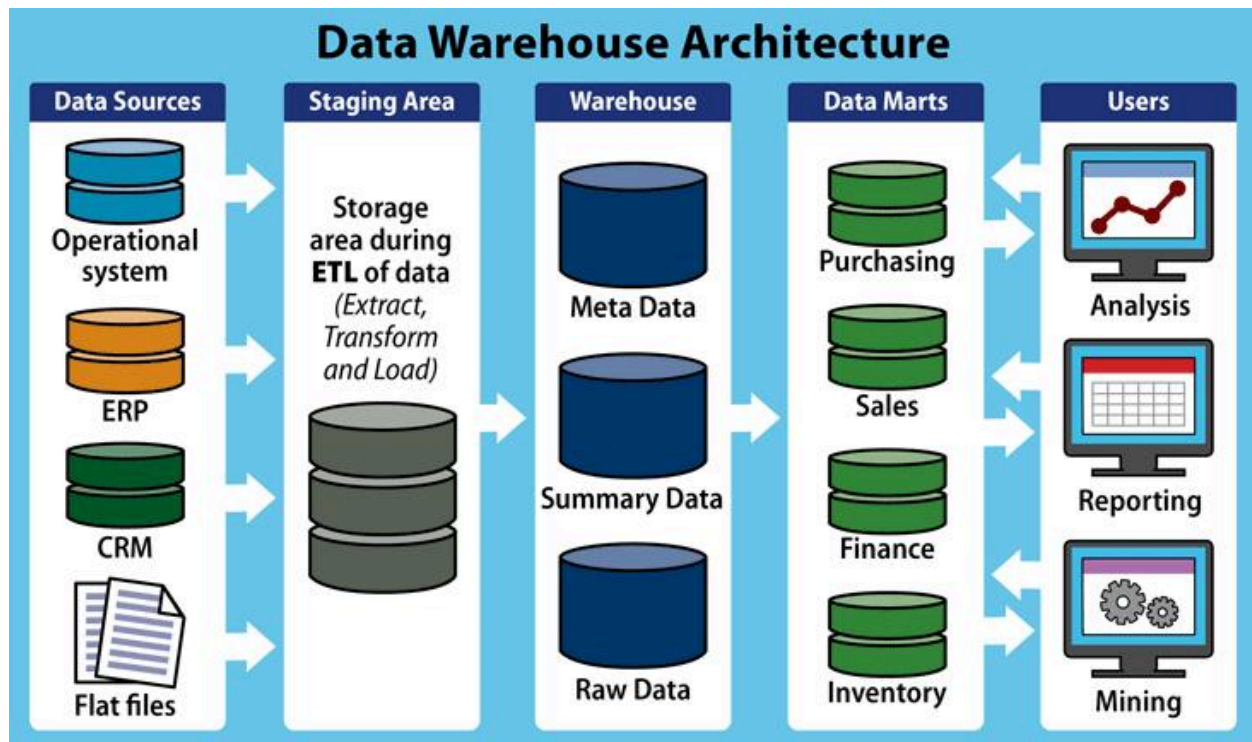
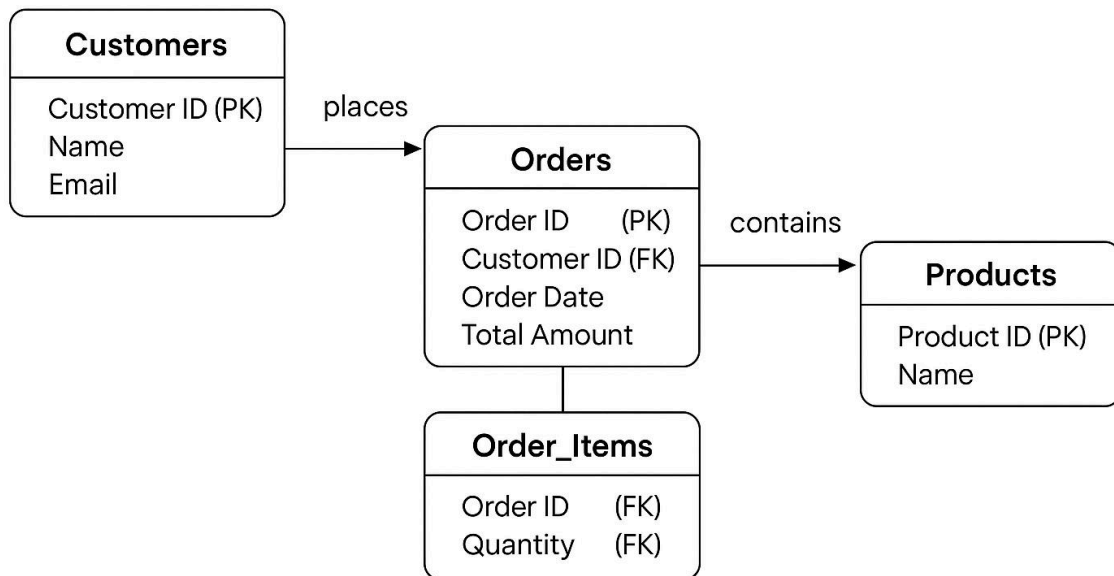
- synchronous calls
- asynchronous calls
- API interactions
- events
- retries
- timeouts
- failure paths
- distributed transactions

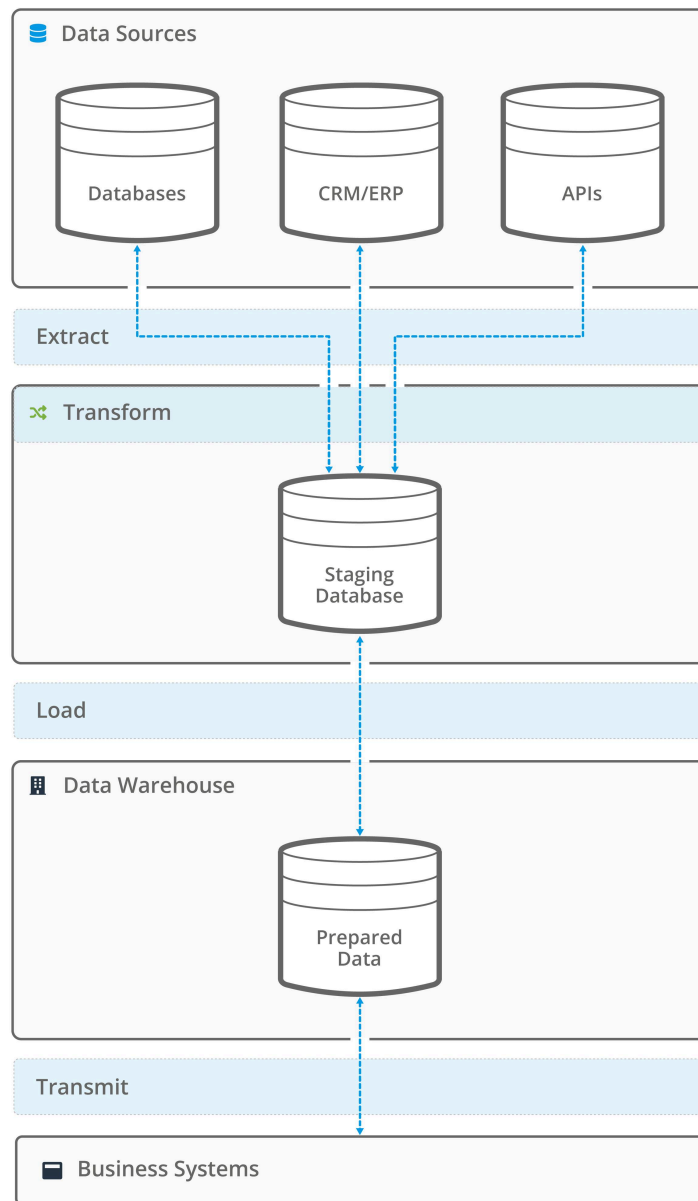
Other useful runtime diagrams

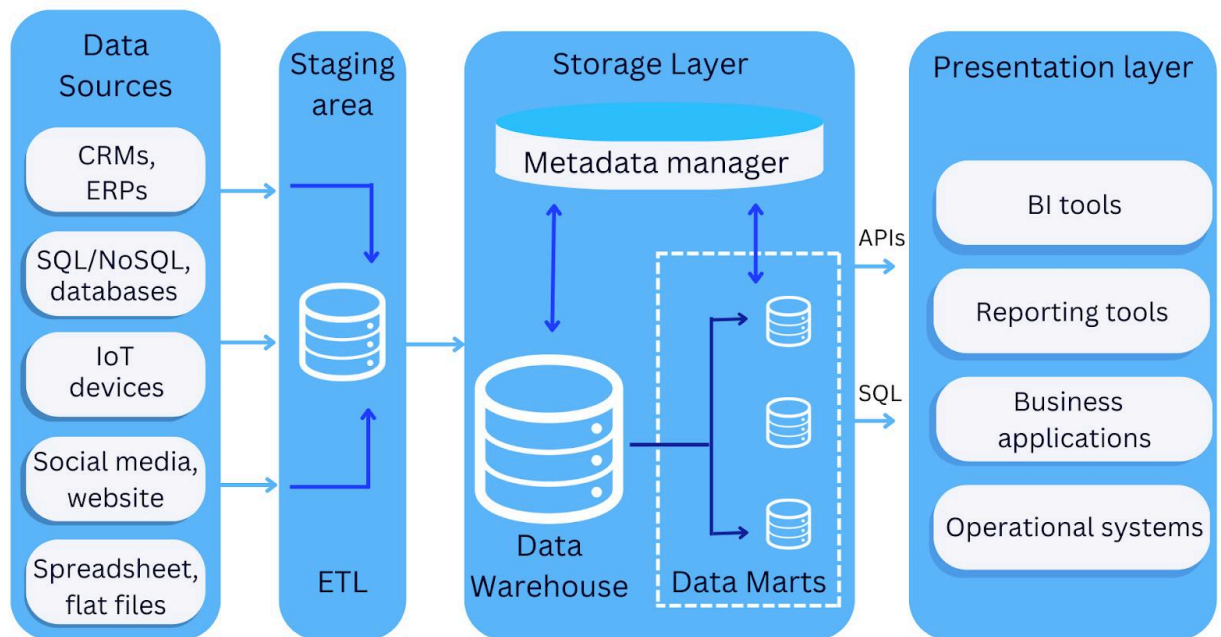
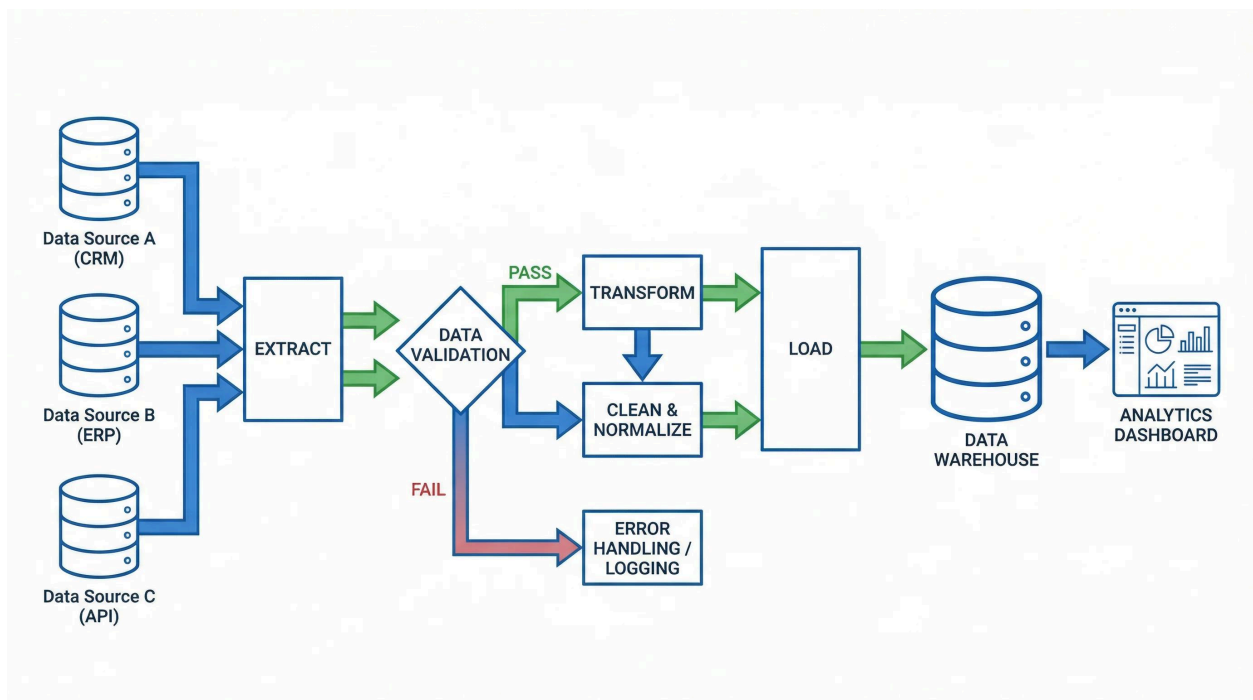
- **Interaction Diagram**
 - **Event Flow Diagram**
 - **Message Flow Diagram**
 - **Request Flow Diagram**
 - **Data Flow Diagram**
 - **Process Flow Diagram**
 - **Swimlane Diagram**
-

4. Data Architecture Diagrams ★★

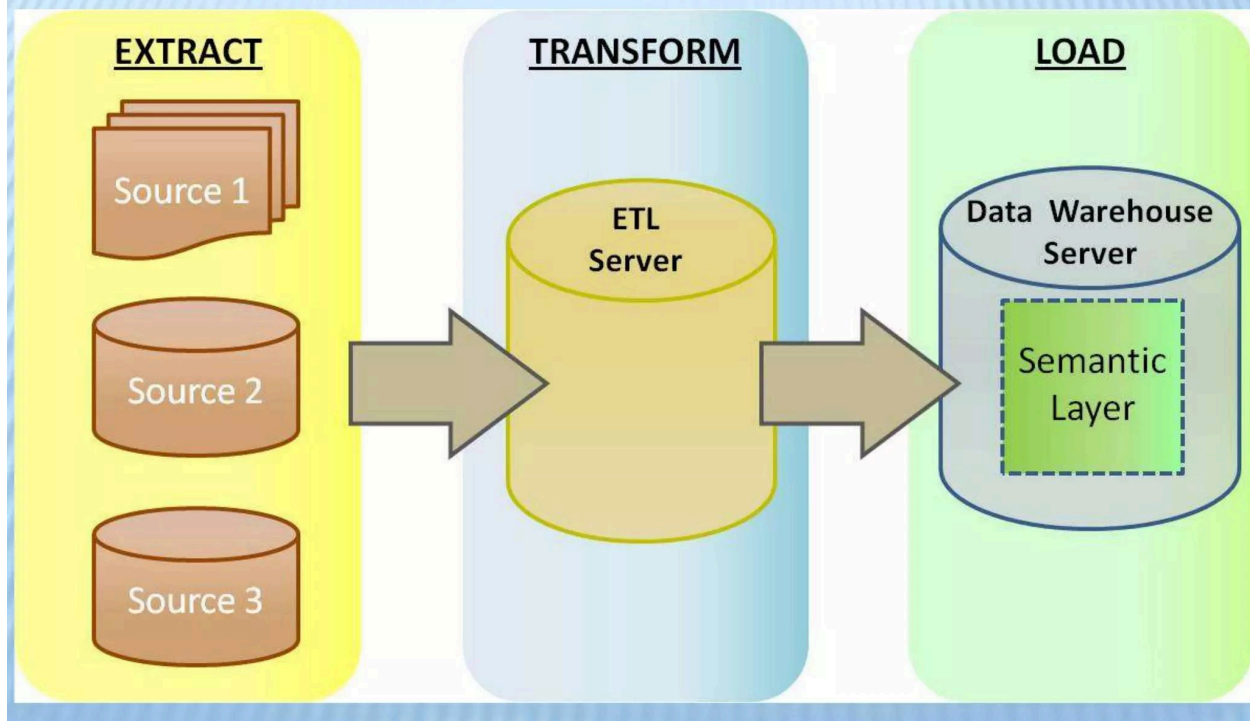
ER Diagram Example







ETL FLOW



An architect should understand:

ERD — Entity Relationship Diagram

Shows:

```
Customer
|
| 1:N
v
Order
|
| 1:N
v
OrderItem
```

Other important data diagrams

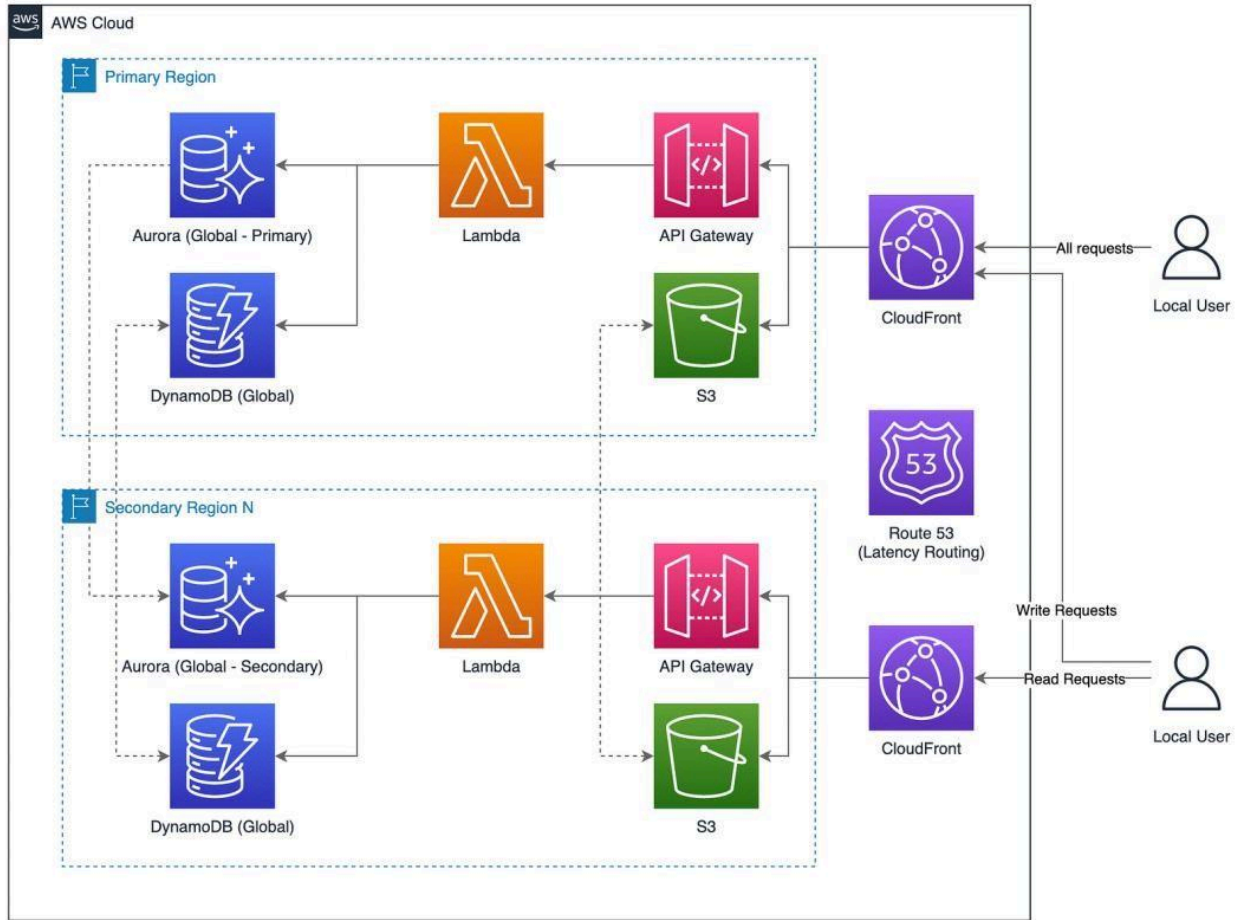
- Logical Data Model
- Physical Data Model
- Conceptual Data Model
- ER Diagram
- Data Flow Diagram (DFD)
- Data Lineage Diagram
- Data Pipeline Diagram
- ETL/ELT Architecture Diagram
- Data Warehouse Architecture
- Data Lake Architecture
- Data Mesh Architecture
- Event/Data Streaming Architecture

For a modern architect, **data lineage** becomes particularly important:

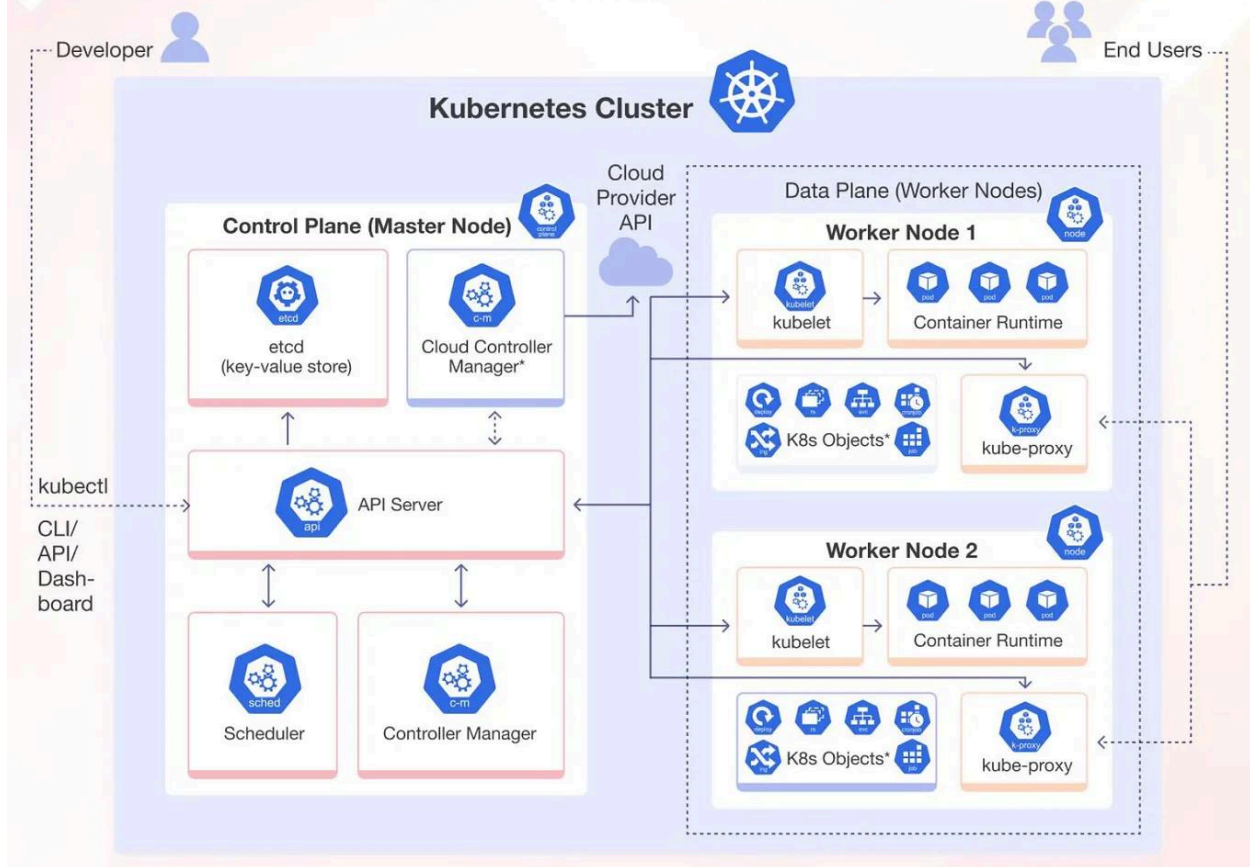
```
graph TD;
  A[Source DB] --> B[CDC];
  B --> C[Kafka];
  C --> D[Data Processing];
  D --> E[Data Warehouse];
  E --> F[BI / Reports];
```

5. Deployment & Infrastructure Diagrams



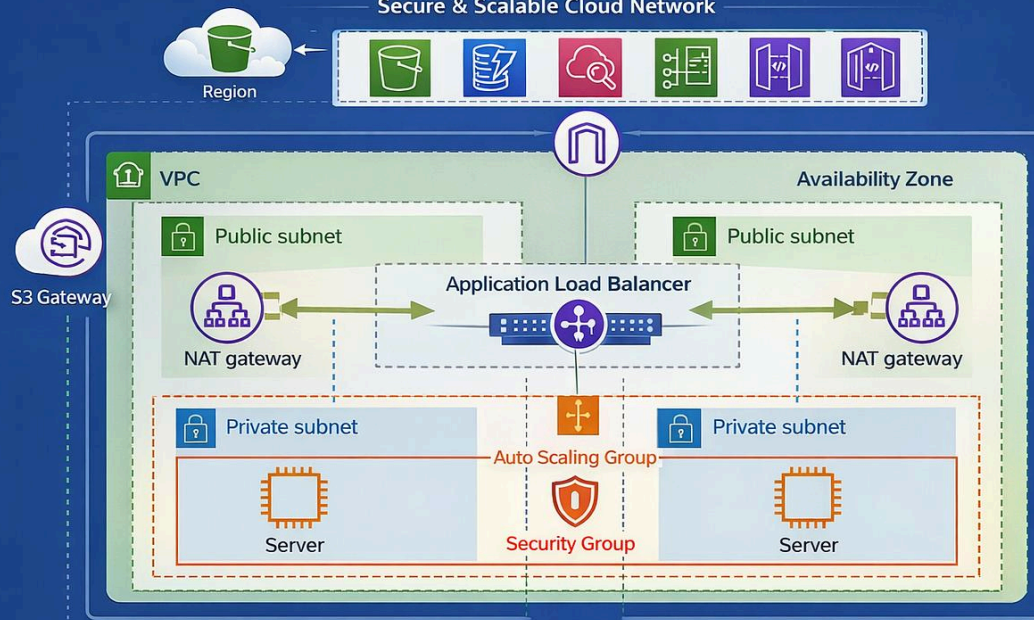


Kubernetes Architecture

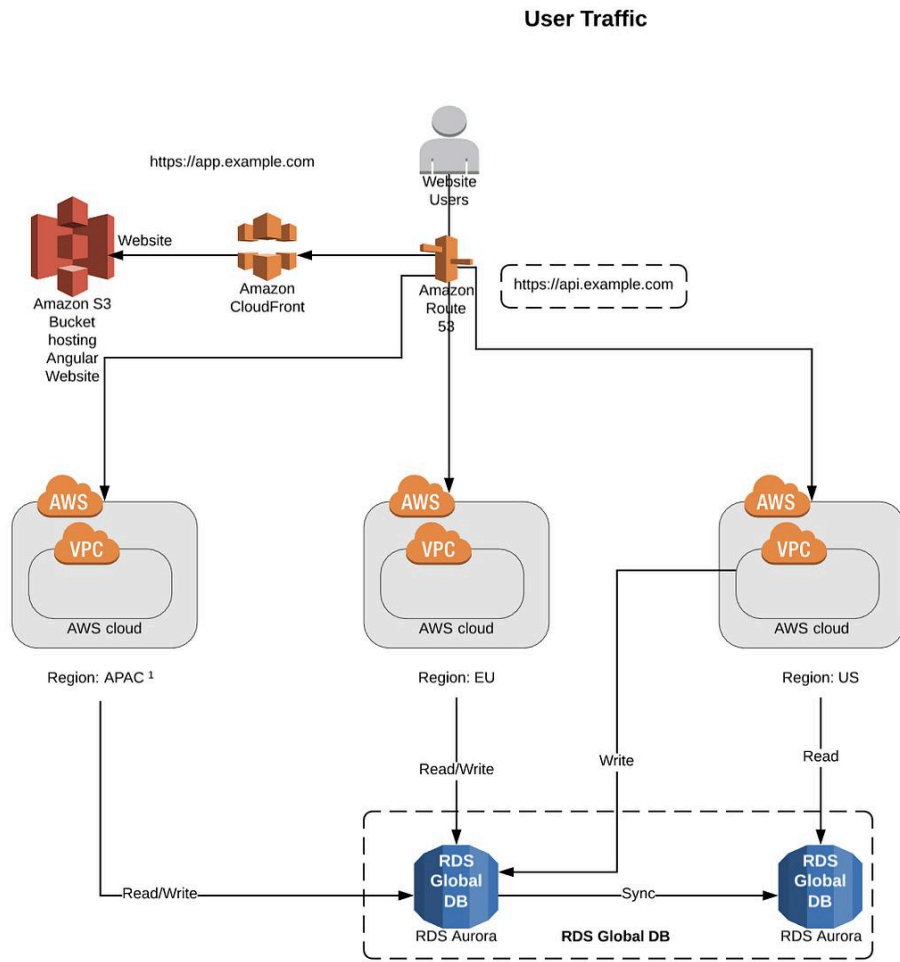


AWS VPC Architecture

Secure & Scalable Cloud Network



Auto Scaling Group | Security Group | NAT Gateway | Public Subnet | Private Subnet



AWS Route 53 Routing Techniques:

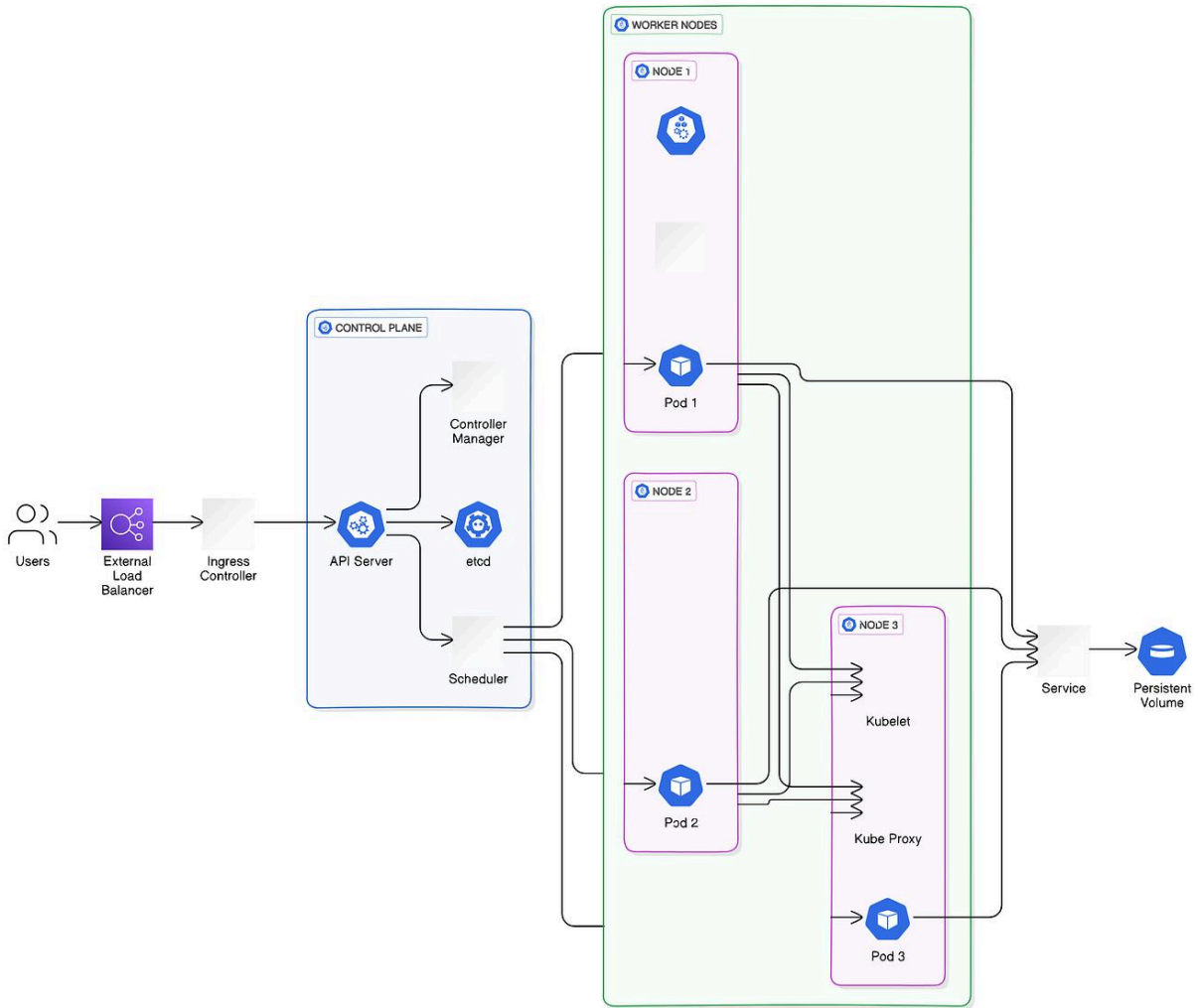
- Geo based
- Latency based
- Failover

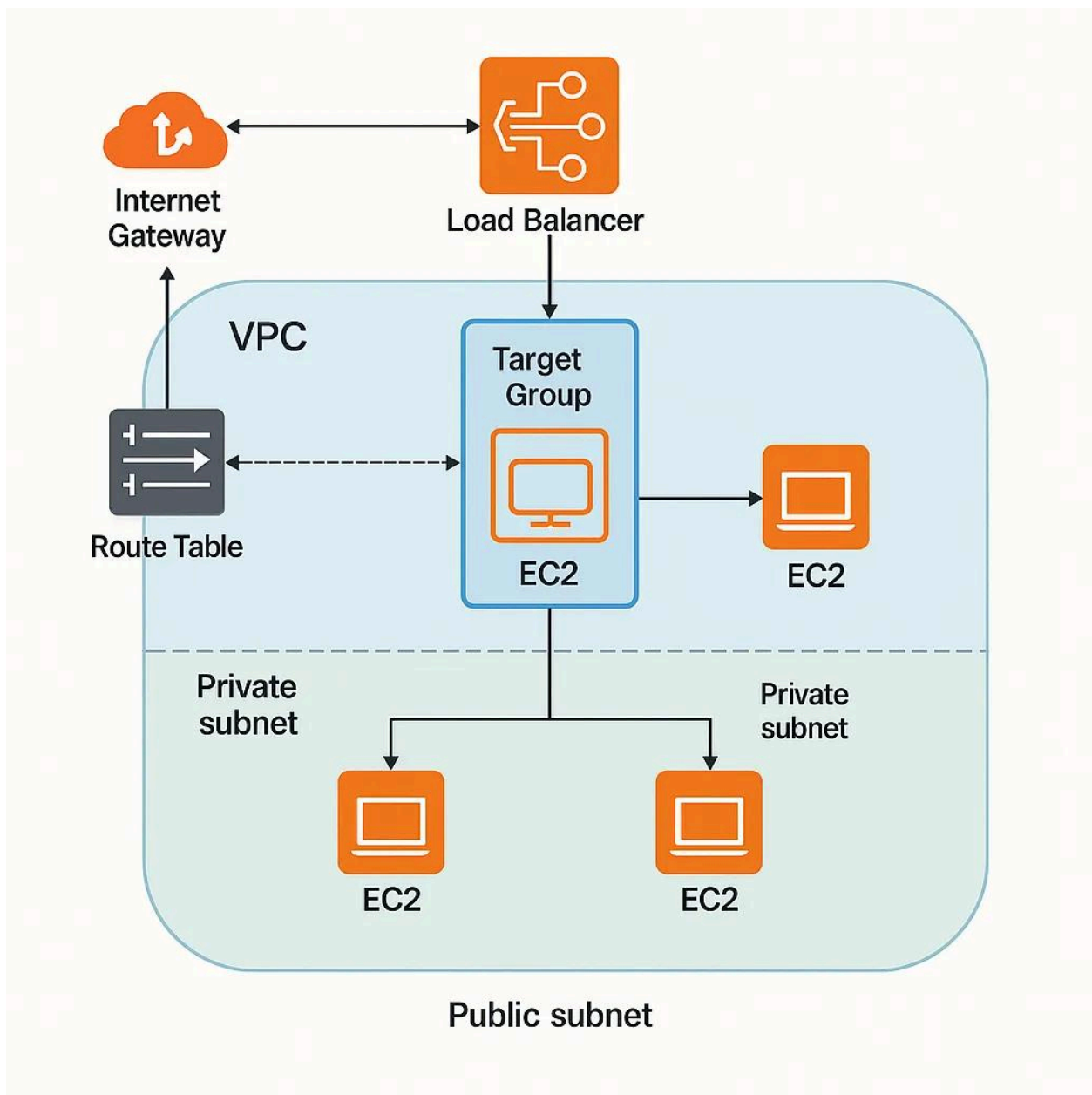
Notes:
 1. Currently, RDS Global DB supports max two regions.

Multi-region Active Active Deployment

@sajidniazi

Kubernetes Architecture Diagram





You should be able to draw:

Deployment Diagram

```

Internet
|
CloudFront
|
Load Balancer
|

```

```
ECS / Kubernetes
|
Microservices
|
Database
```

Infrastructure diagrams

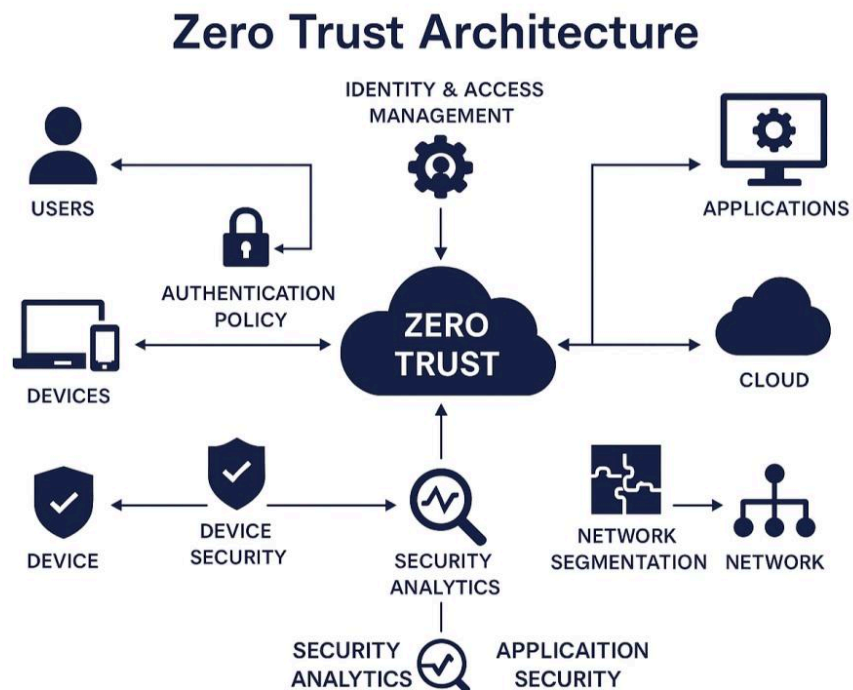
- **Network Diagram**
- **VPC Diagram**
- **Subnet Diagram**
- **Kubernetes Architecture Diagram**
- **Container Architecture Diagram**
- **Server Architecture Diagram**
- **Cloud Infrastructure Diagram**
- **Multi-region Architecture**
- **DR Architecture**
- **High Availability Architecture**
- **Disaster Recovery Diagram**

Important concepts to represent:

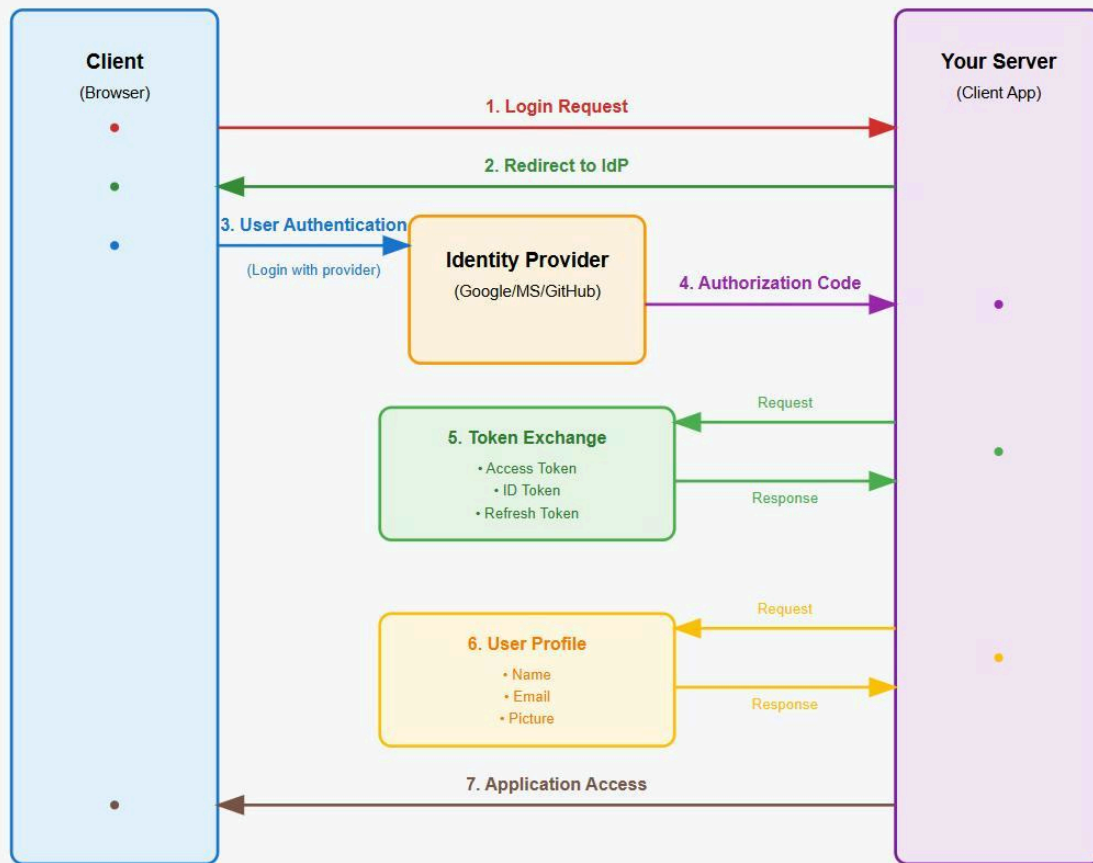
- Availability Zones
- Regions
- Load balancers
- CDN
- DNS
- WAF
- firewalls
- private/public networks
- databases

- queues
- caches
- object storage

6. Security Architecture Diagrams ★★ ★



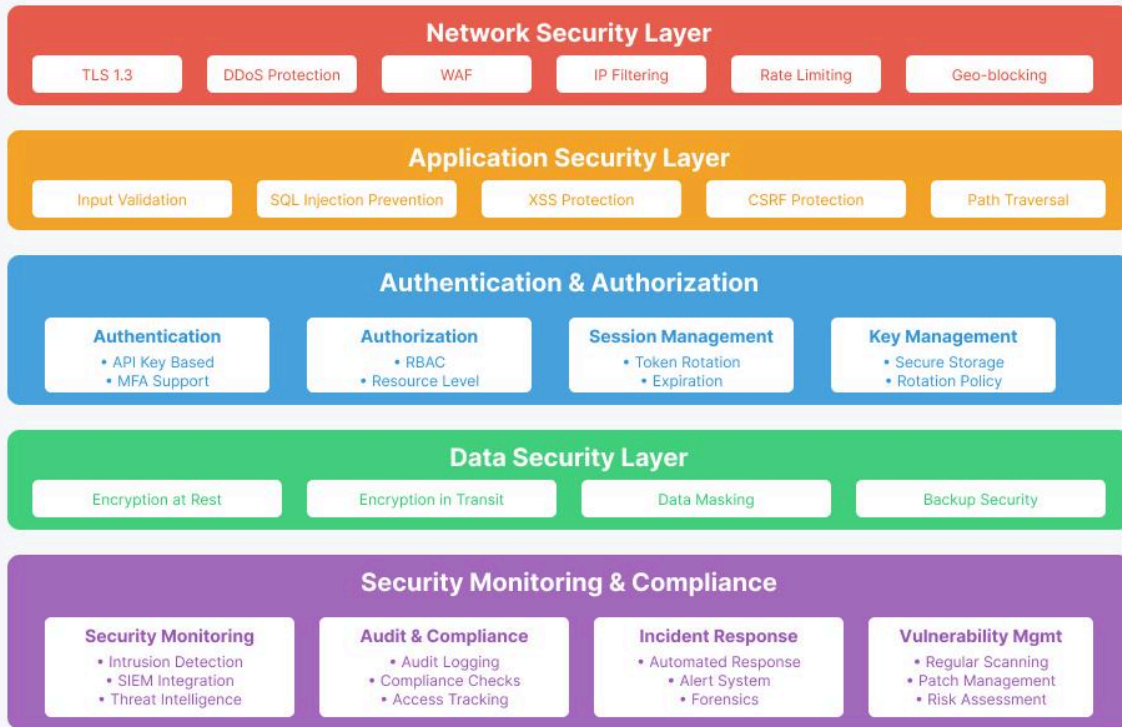
OAuth 2.0/OpenID Connect Authentication Architecture



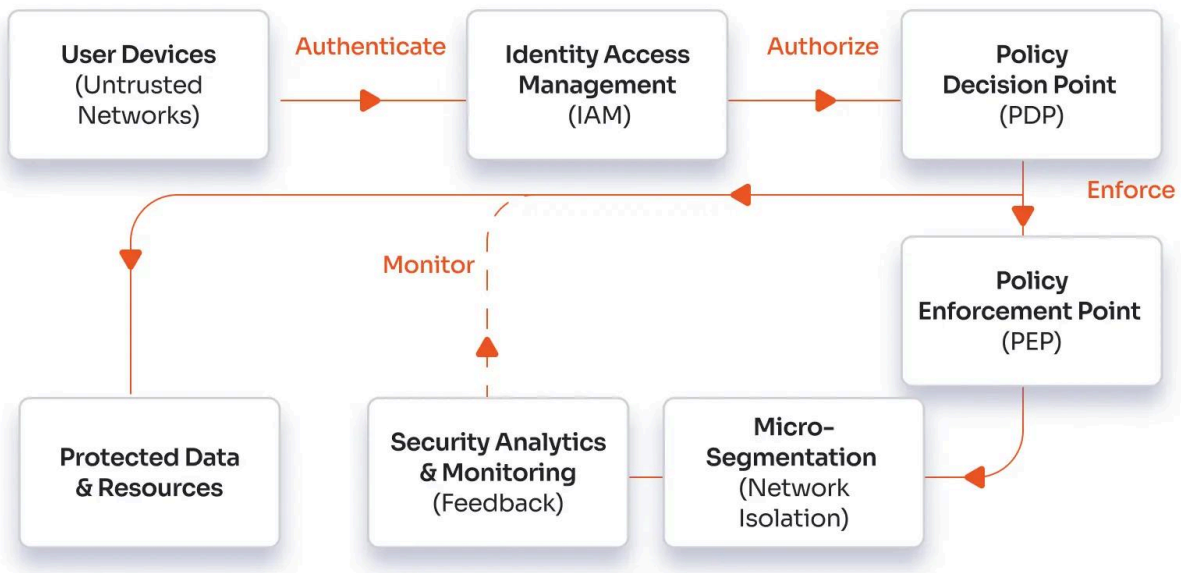
OAuth 2.0/OpenID Connect Authentication Flow:

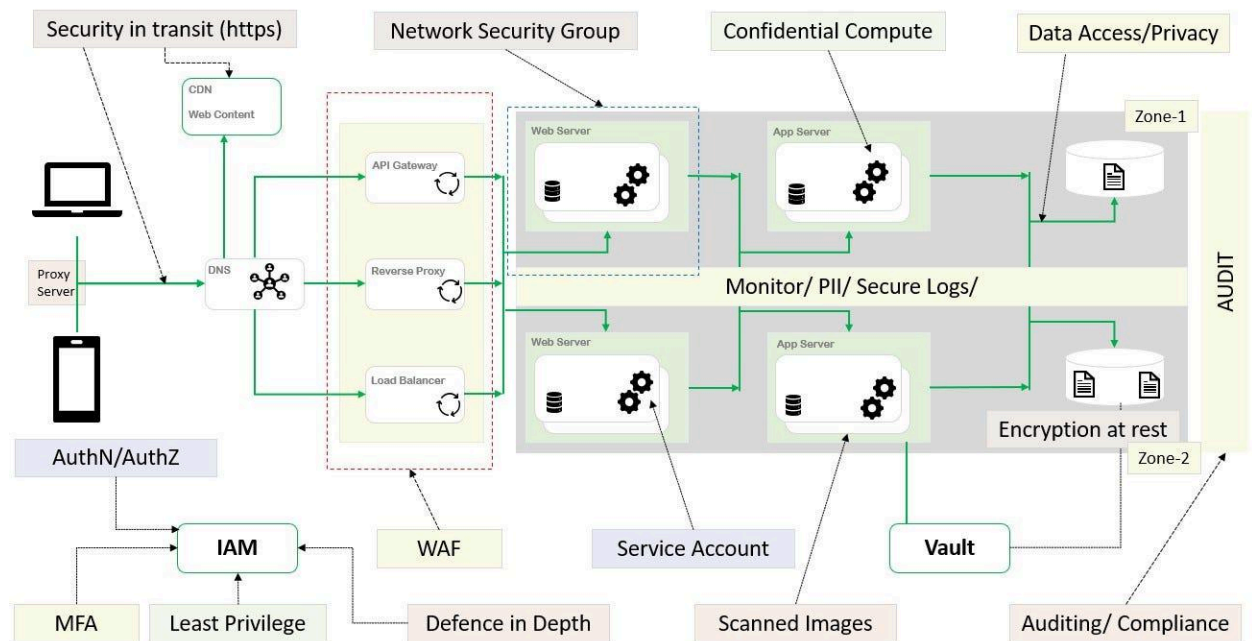
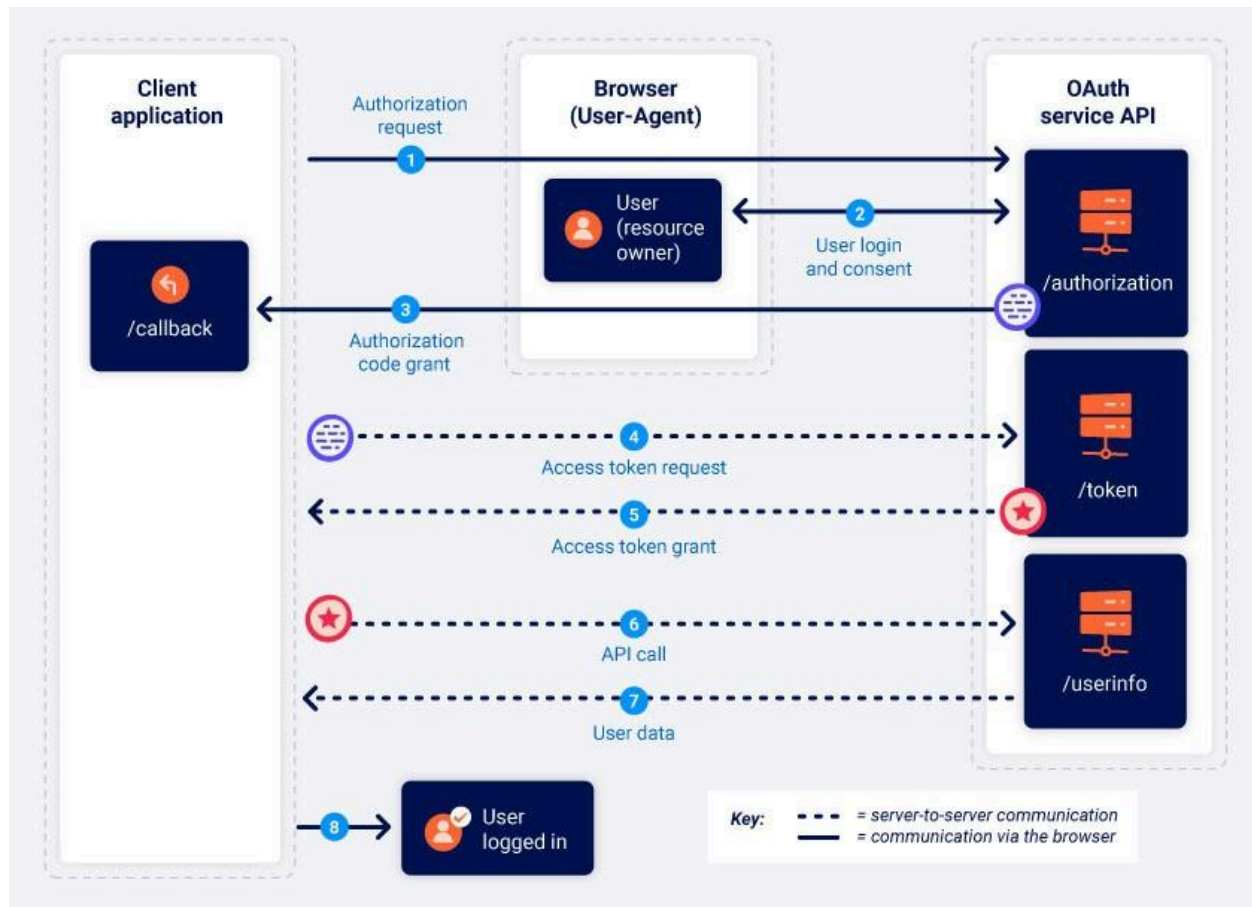
1. Client requests login → 2. Server redirects to Identity Provider → 3. User authenticates with IdP → 4. IdP returns authorization code
5. Server exchanges code for tokens (bidirectional) → 6. Server fetches user profile (bidirectional) → 7. Server grants application access

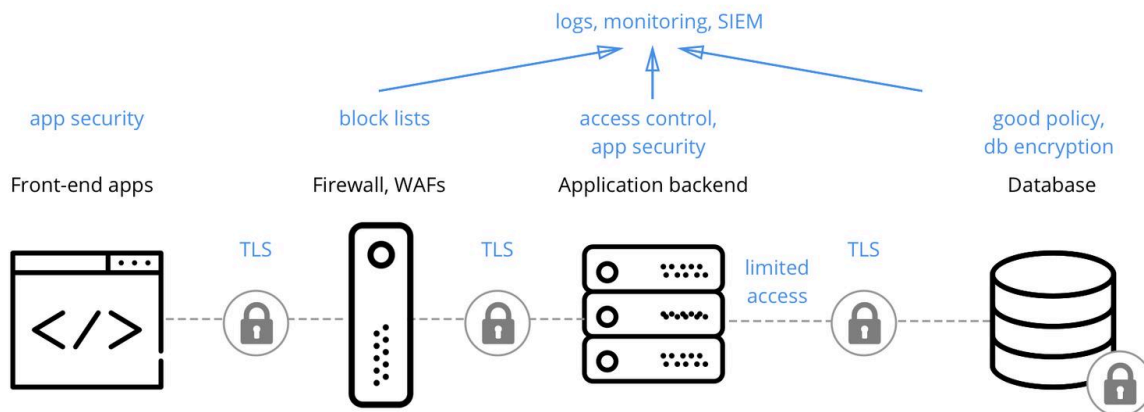
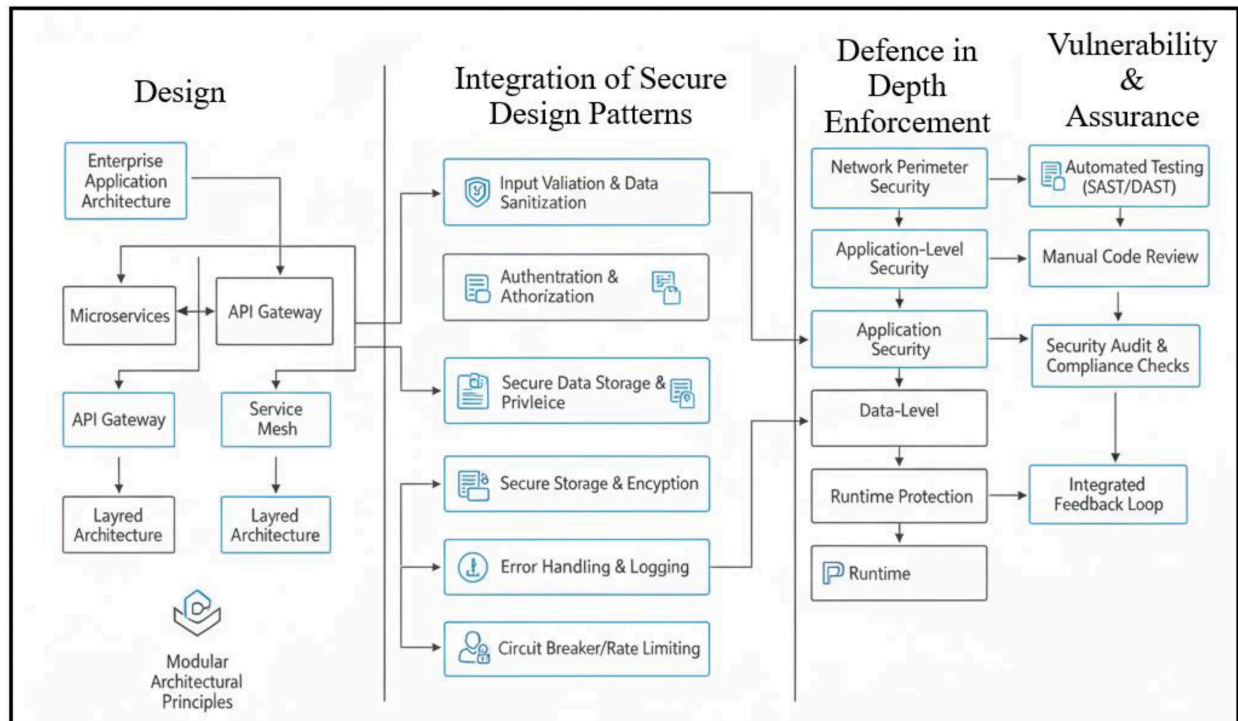
Security Architecture



Zero trust Architecture







An architect should know:

- **Security Architecture Diagram**
- **Trust Boundary Diagram**
- **Threat Model Diagram**
- **Data Classification Diagram**
- **Authentication Flow Diagram**
- **Authorization Flow Diagram**

- **OAuth 2.0 Flow**
- **OIDC Flow**
- **Zero Trust Architecture**
- **Network Security Diagram**
- **Identity Architecture**
- **Encryption / Key Management Diagram**

For example:

```

User
↓
Identity Provider
↓
OAuth/OIDC
↓
API Gateway
↓
Authorization
↓
Microservice

```

7. Business & Domain Diagrams

These are often overlooked by developers but are **very important for architects**.

Domain Model

```

Customer
|
+--- Account
|
+--- Order
      |
      +--- Payment

```

Other diagrams

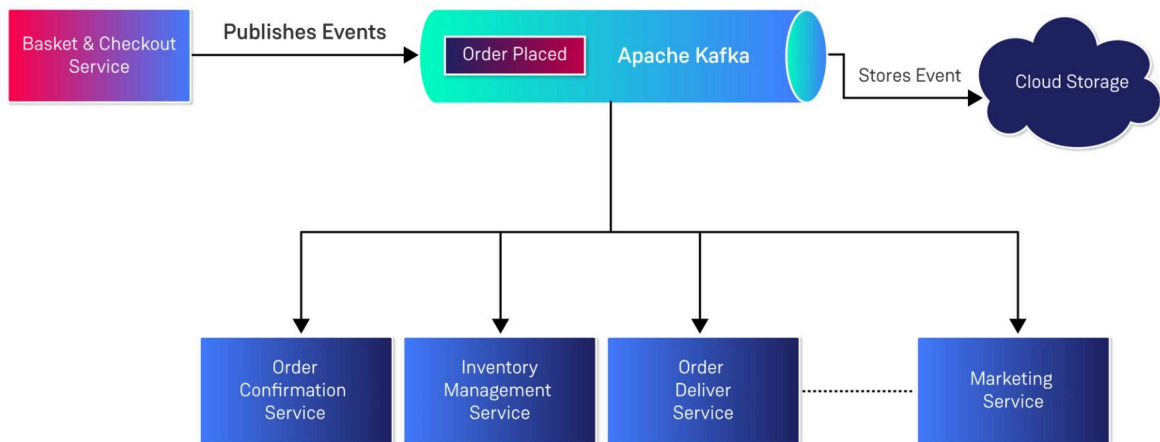
- **Domain Model**
- **Domain Map**
- **Bounded Context Map** ★
- **Context Mapping Diagram** ★
- **Business Capability Map**
- **Business Process Diagram**
- **Value Stream Map**
- **Organization/System Landscape**
- **Event Storming Model**

For **DDD**, I would definitely know:

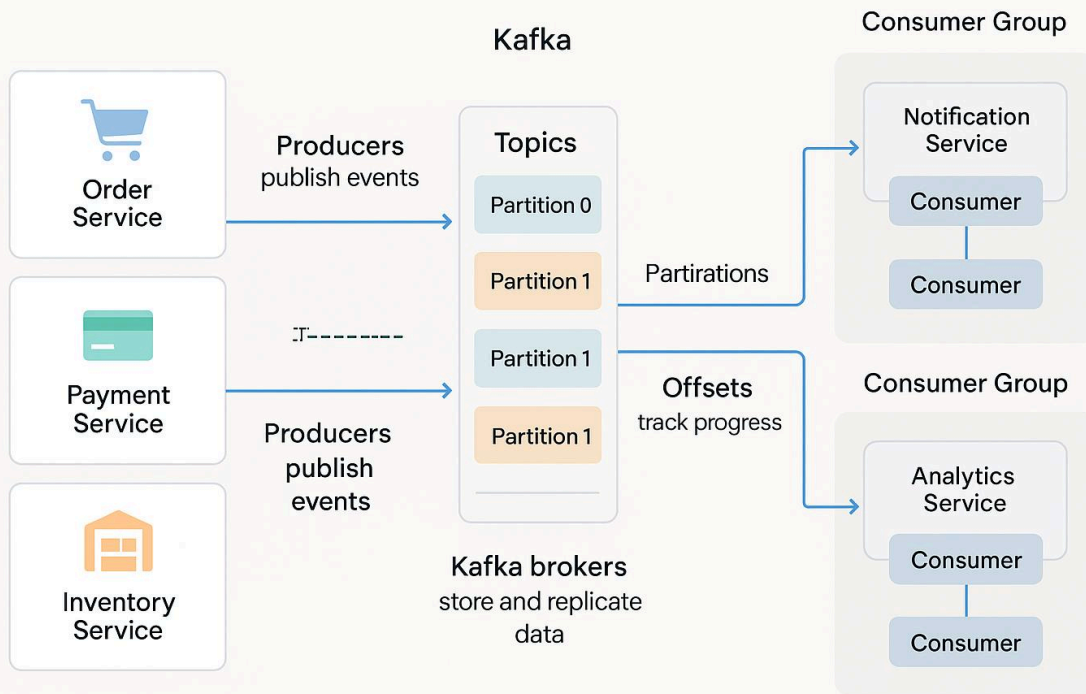
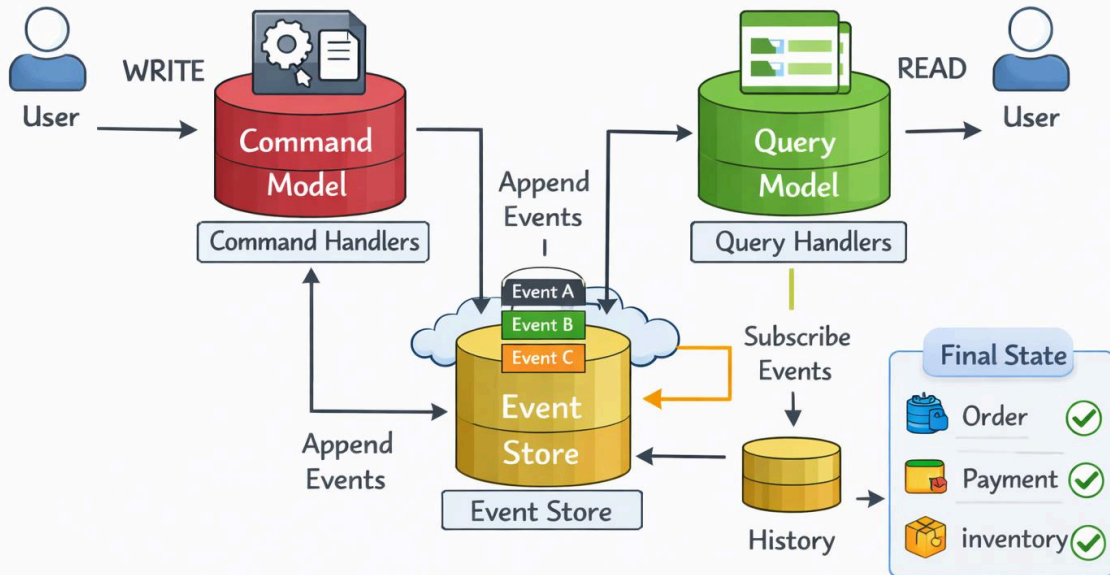
| **Bounded Context Map + Domain Model + Event Storming**

8. Event-Driven Architecture Diagrams



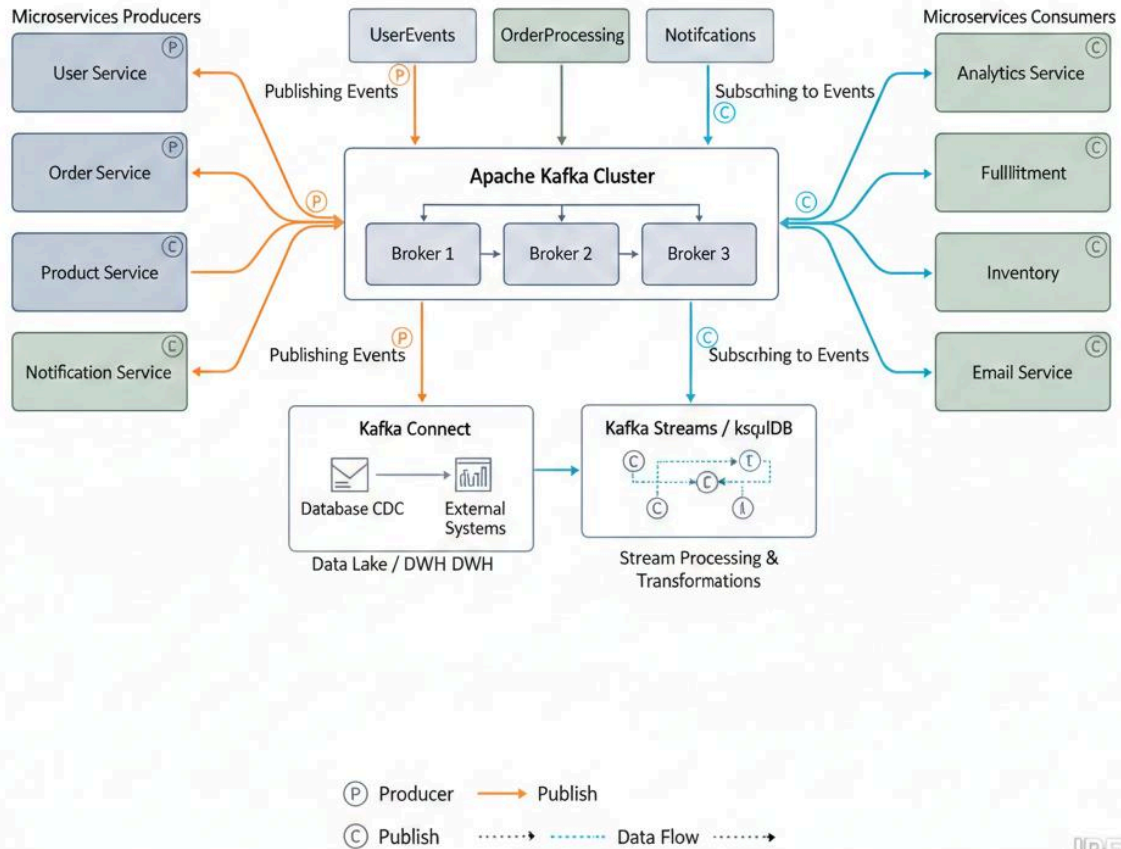


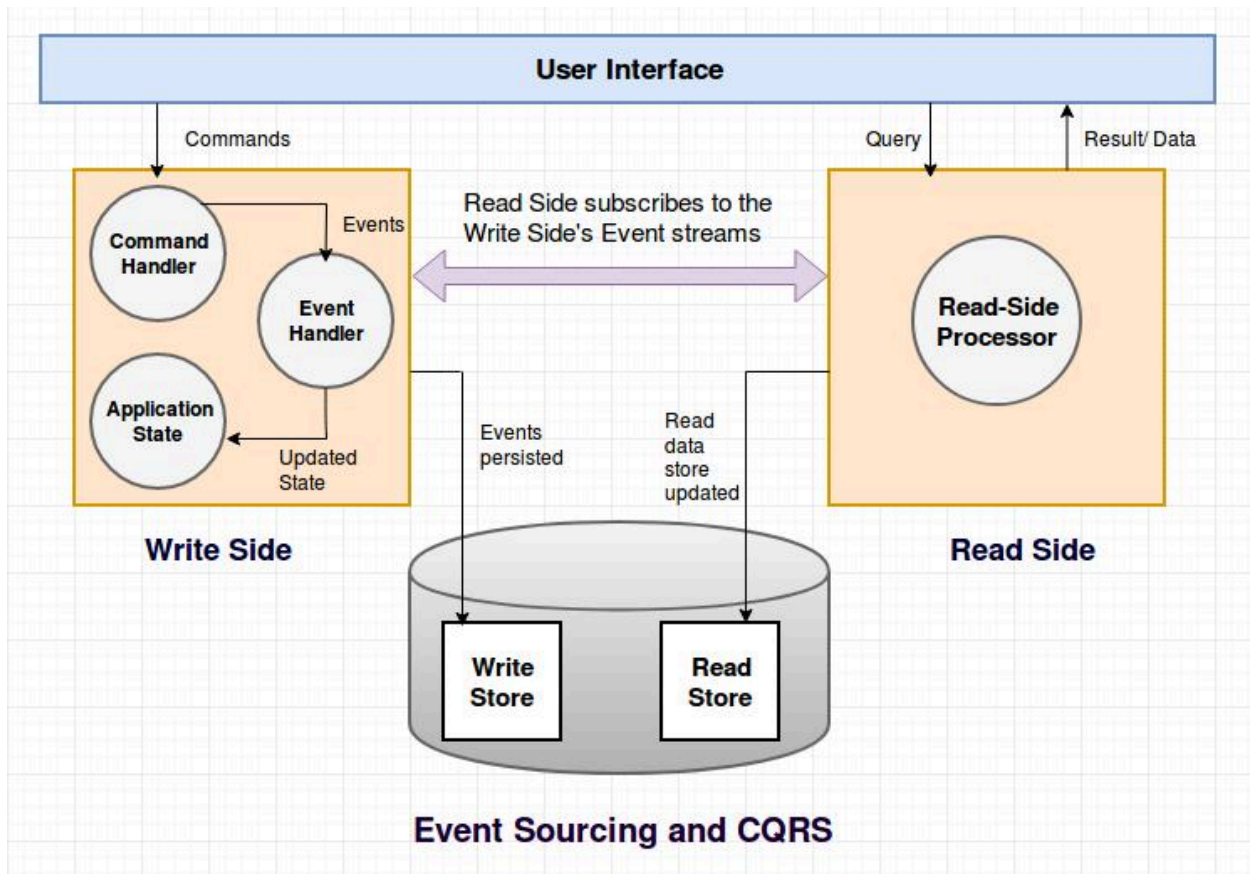
CQRS with Event Sourcing

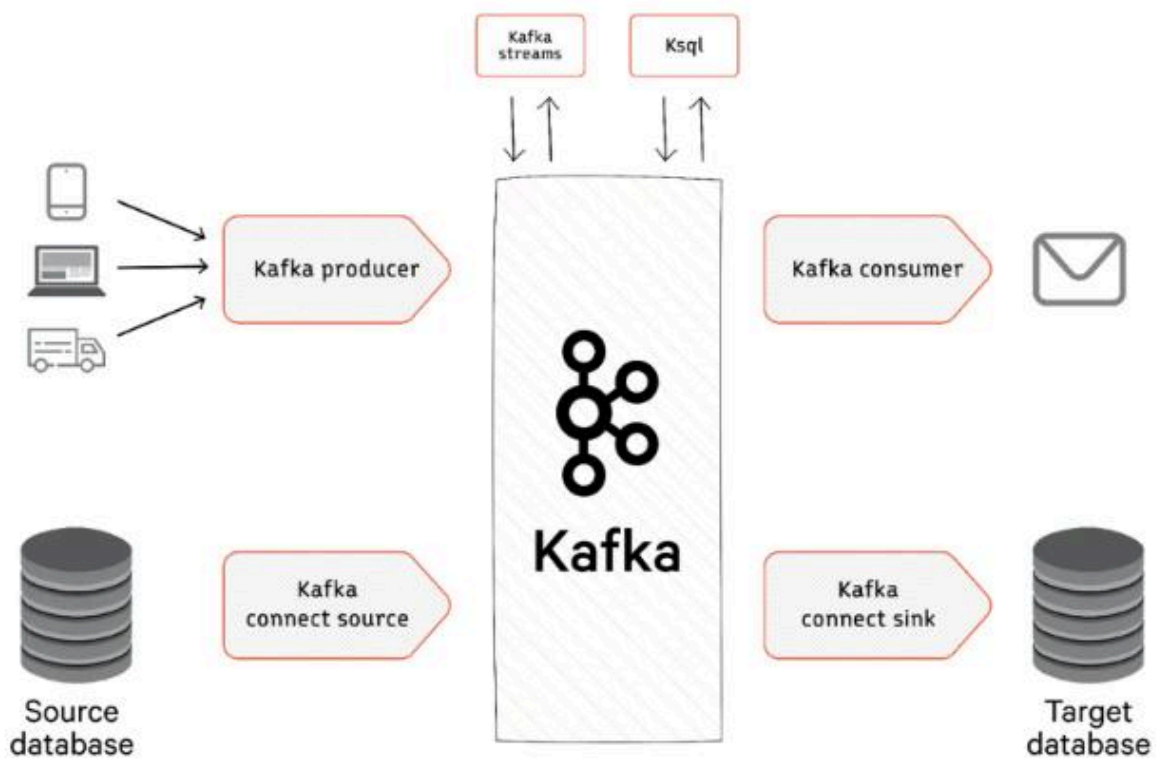


Microservices Architecture with Apache Kafka

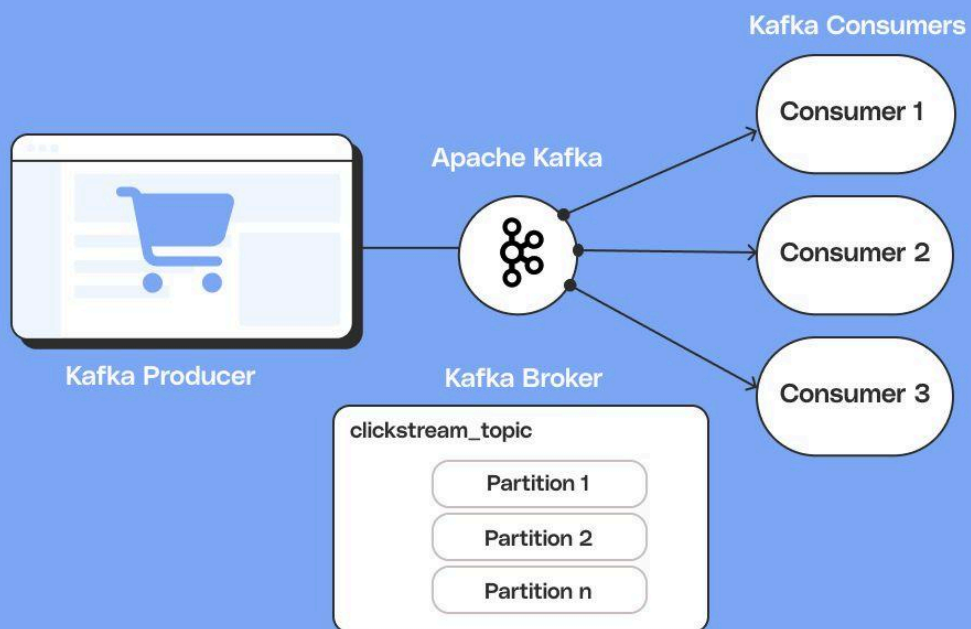
Event-Driven Data Flow and Integration







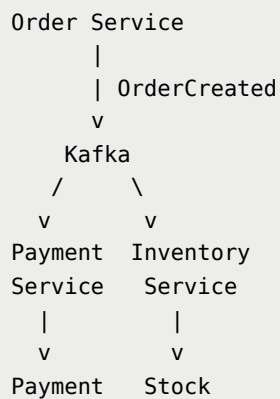
E-commerce event-driven architecture



Important for modern distributed systems:

- **Event Flow Diagram**
- **Event Storming**
- **Event Topology Diagram**
- **Message Flow Diagram**
- **Kafka Topic Architecture**
- **CQRS Diagram**
- **Event Sourcing Diagram**
- **Saga Diagram**
- **Outbox Pattern Diagram**
- **Pub/Sub Diagram**

Example:



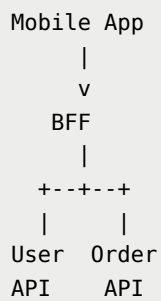
9. Integration & API Diagrams ★★

Useful when designing enterprise systems.

- **API Architecture Diagram**
- **API Gateway Diagram**
- **Integration Architecture**

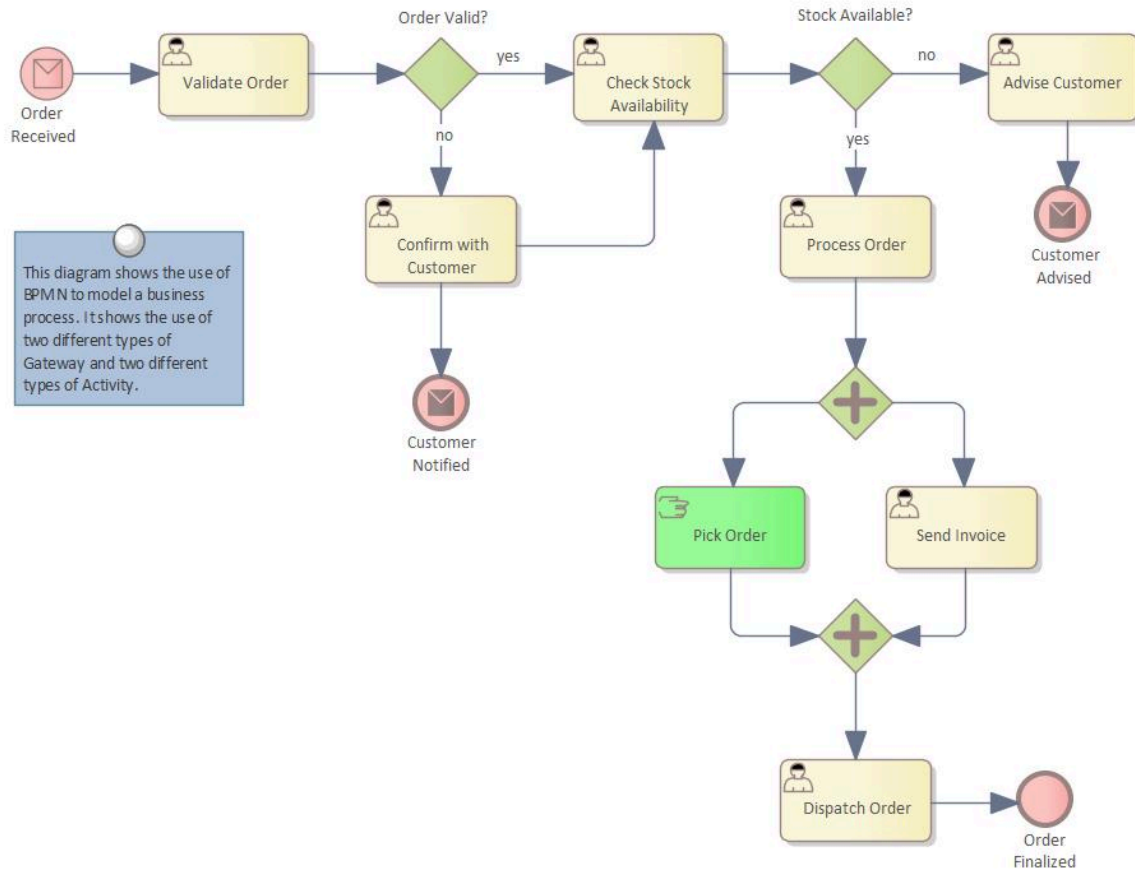
- **System Integration Diagram**
- **Sequence Diagram**
- **Message Flow Diagram**
- **ESB Architecture**
- **Webhook Flow**
- **BFF Architecture**
- **Service Mesh Diagram**

For example:

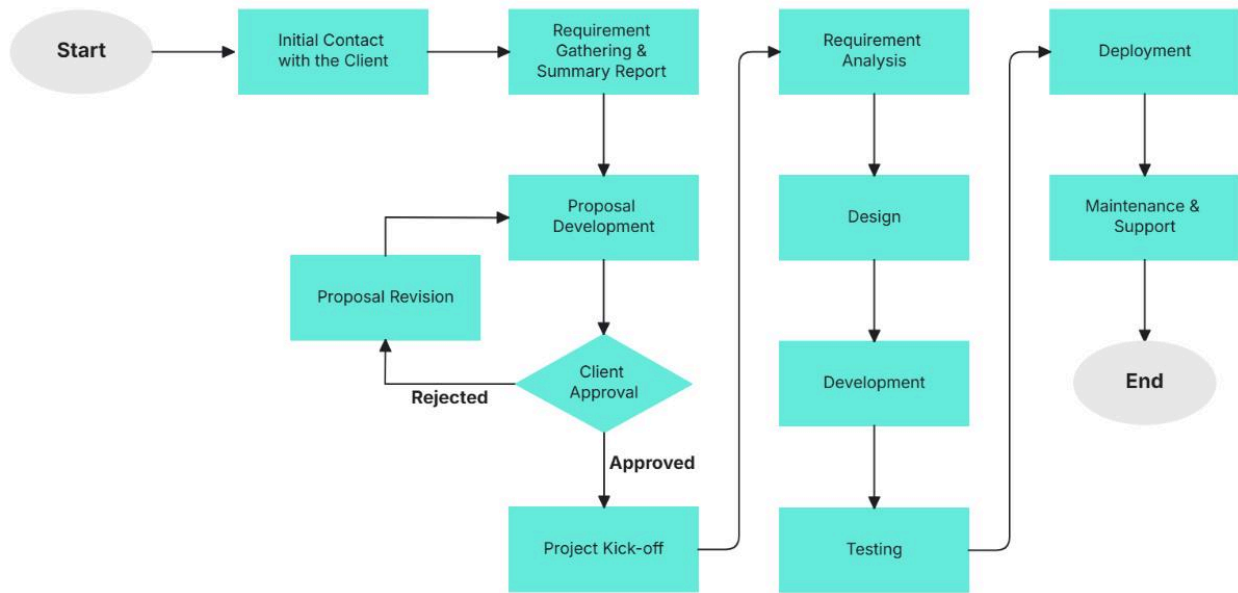


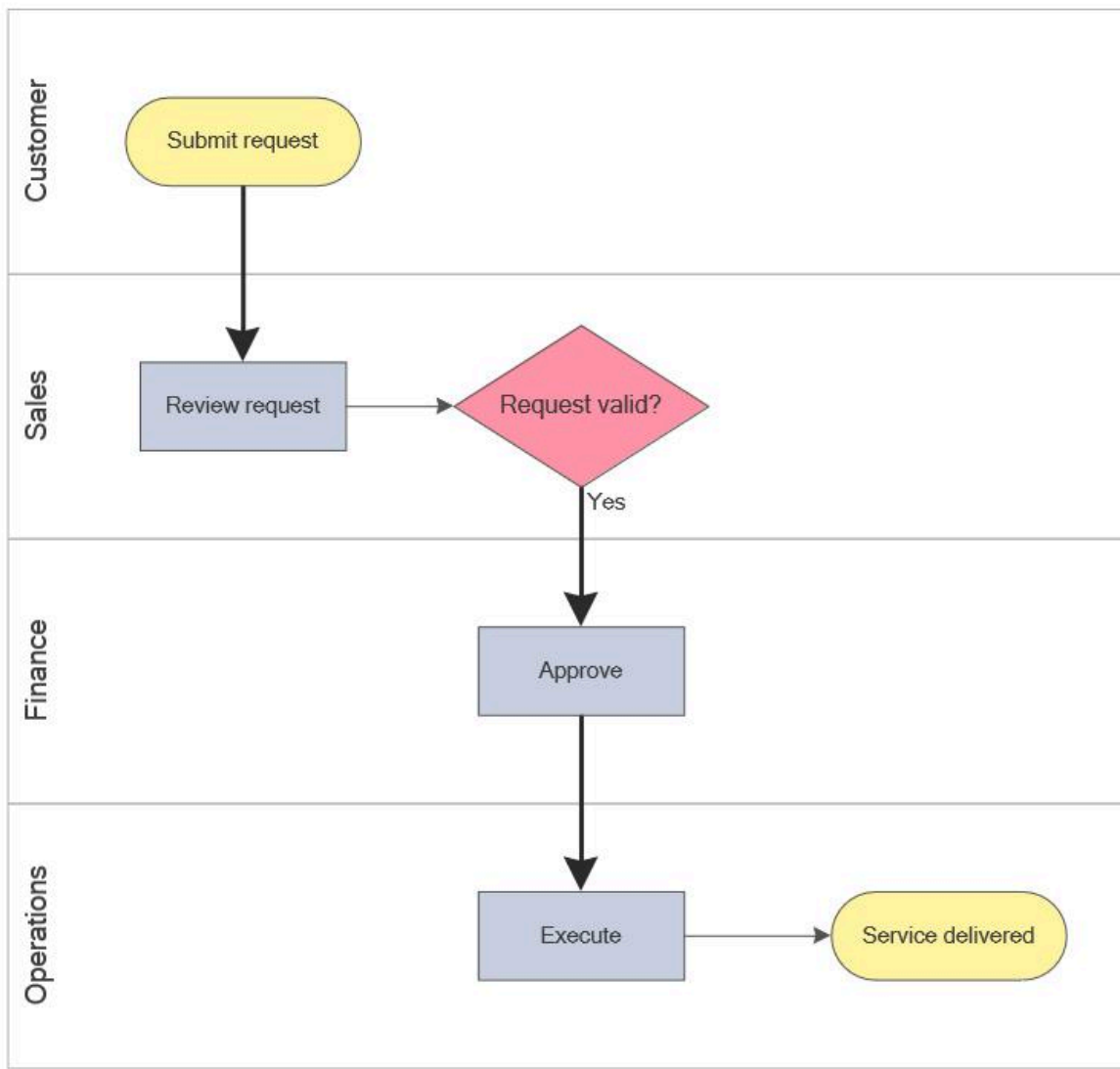
10. Process & Workflow Diagrams ★★

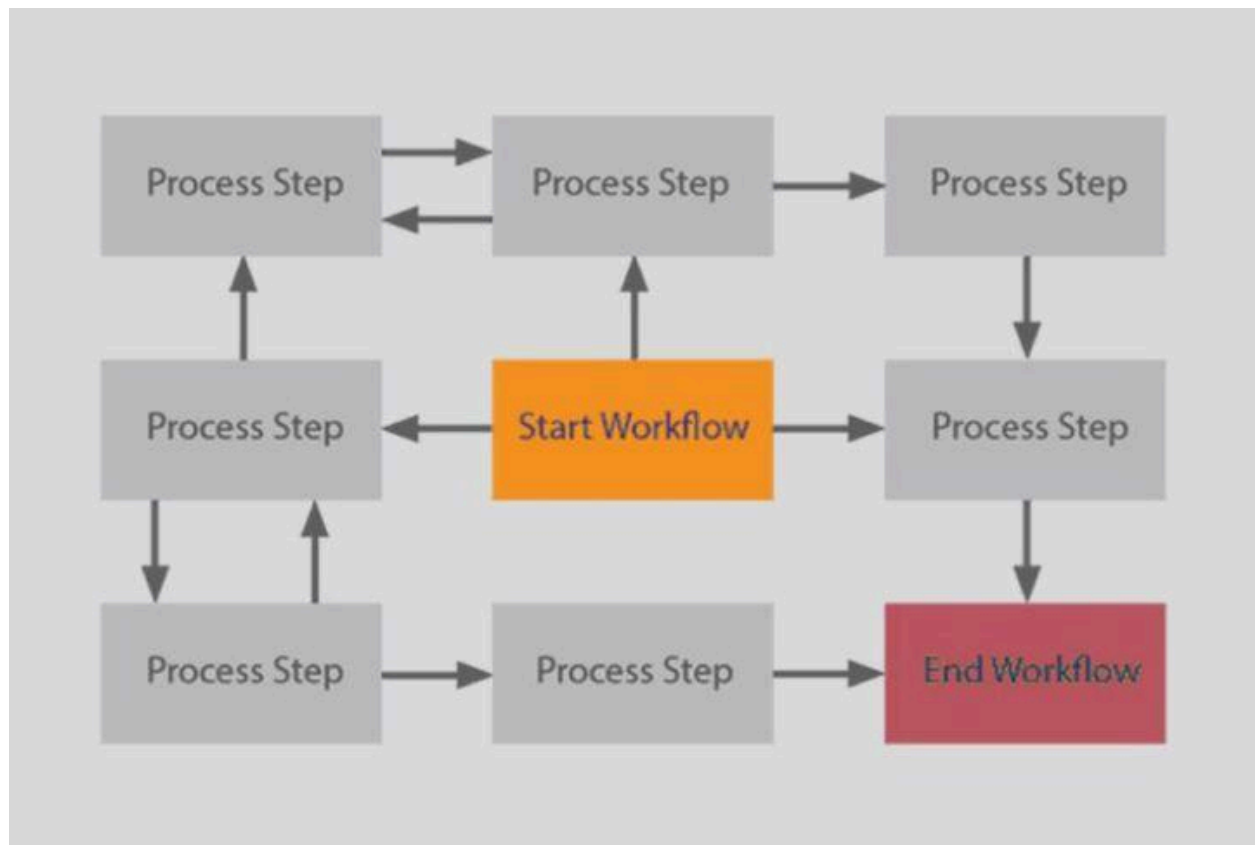
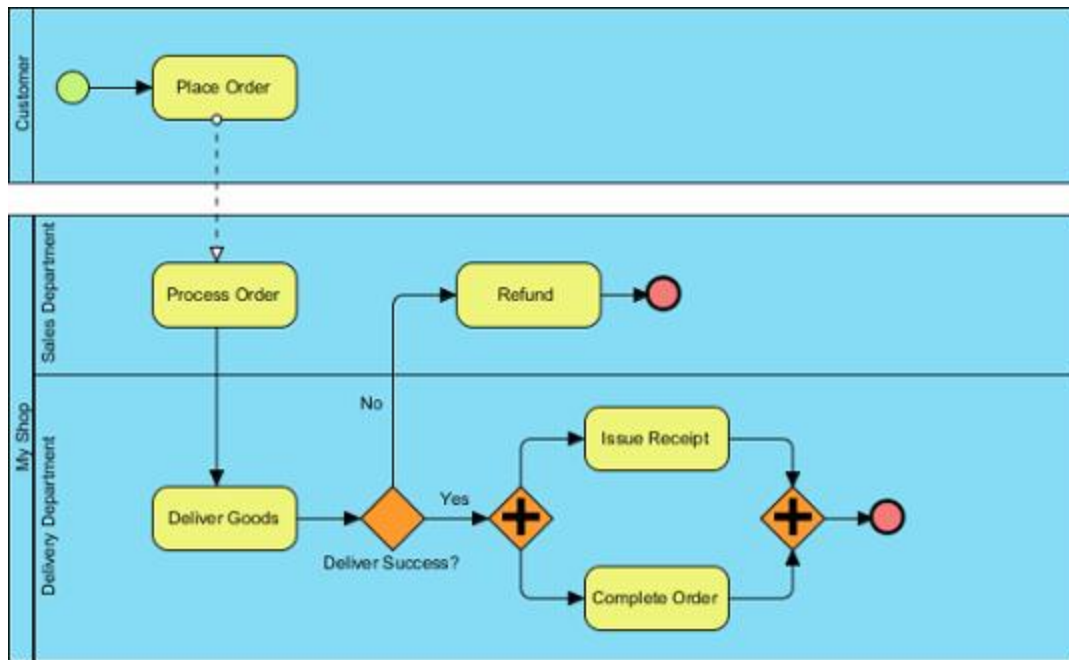
Business Process Order Management

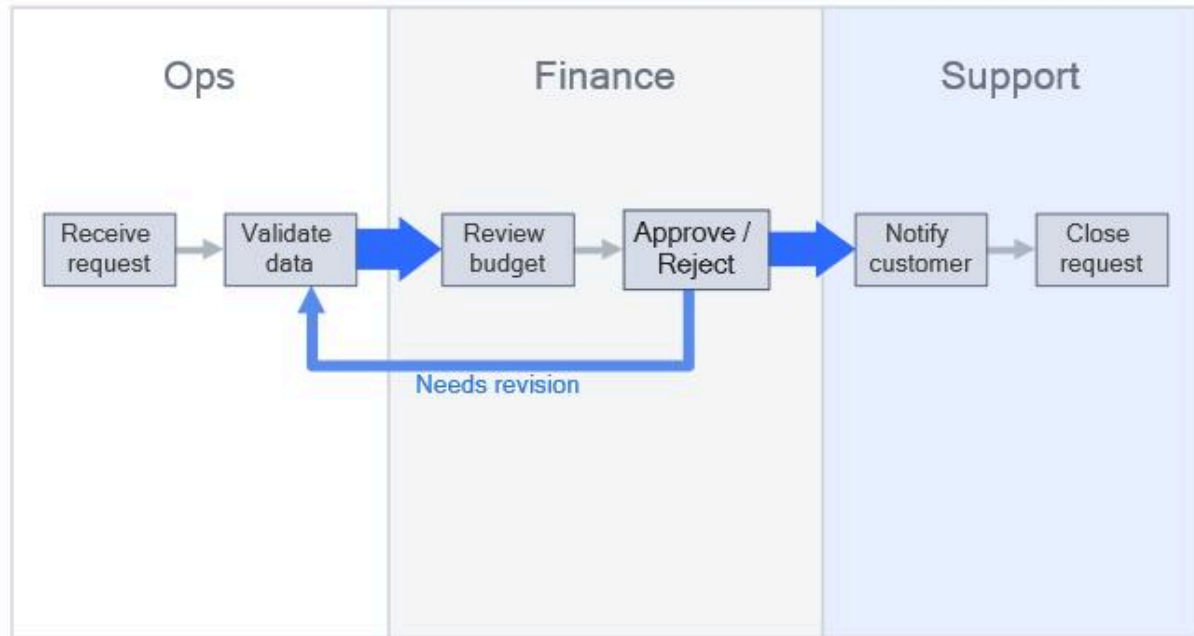


Business Process Workflow Template







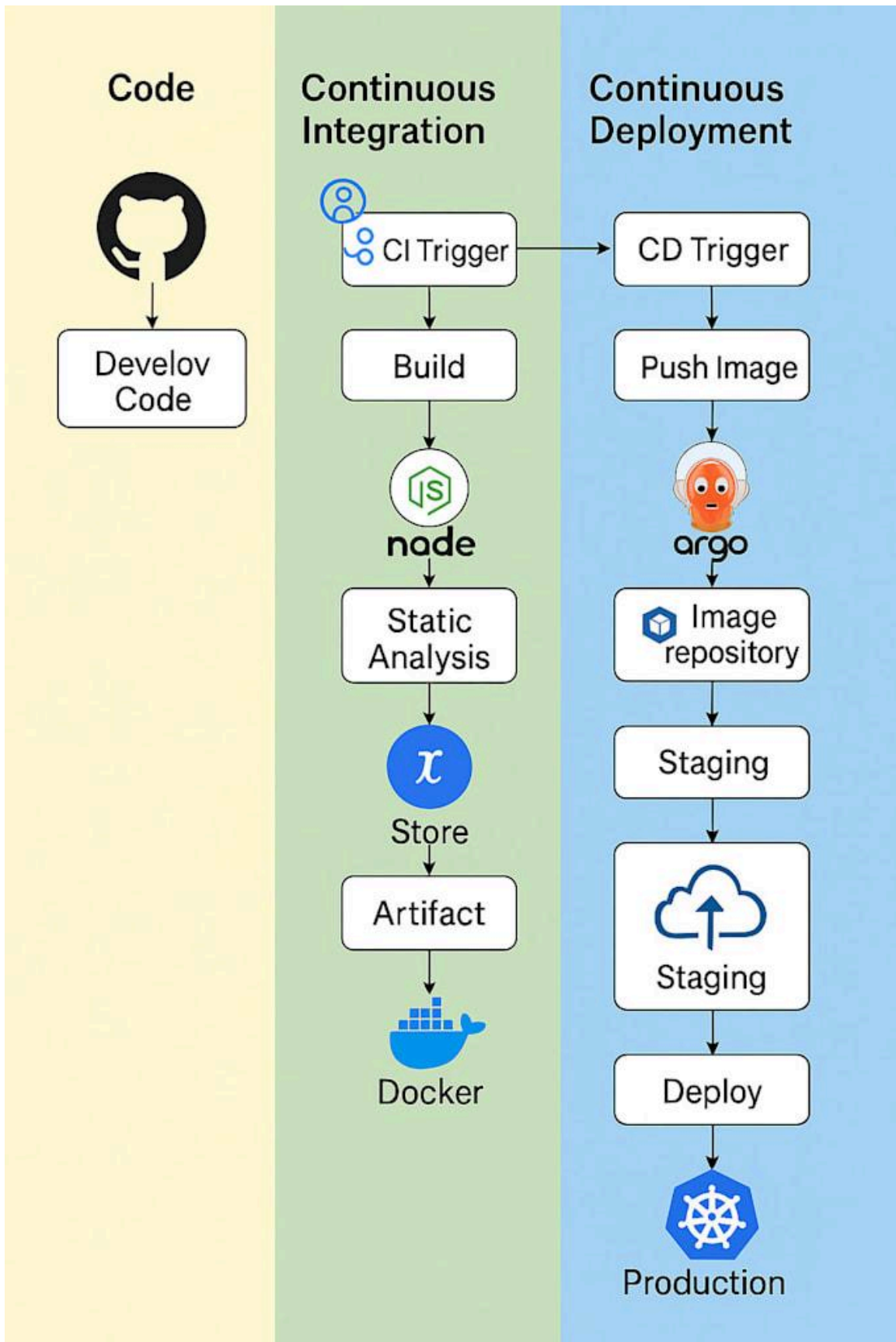


Know:

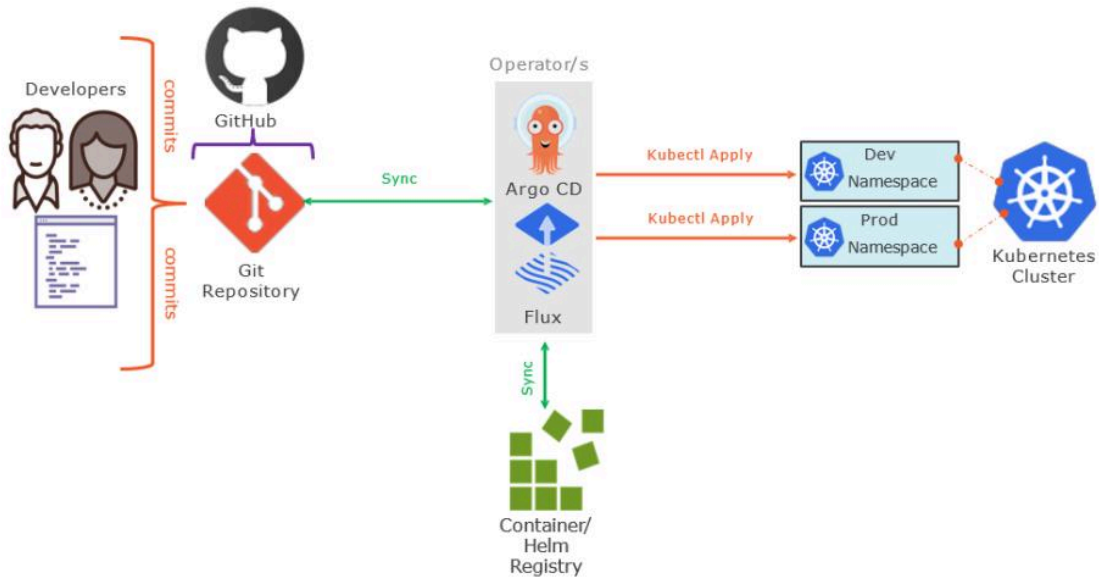
- **Flowchart**
- **Swimlane Diagram**
- **BPMN**
- **Workflow Diagram**
- **Activity Diagram**
- **Process Flow Diagram**
- **Decision Tree**
- **State Transition Diagram**

BPMN is particularly useful when working with business analysts and enterprise stakeholders.

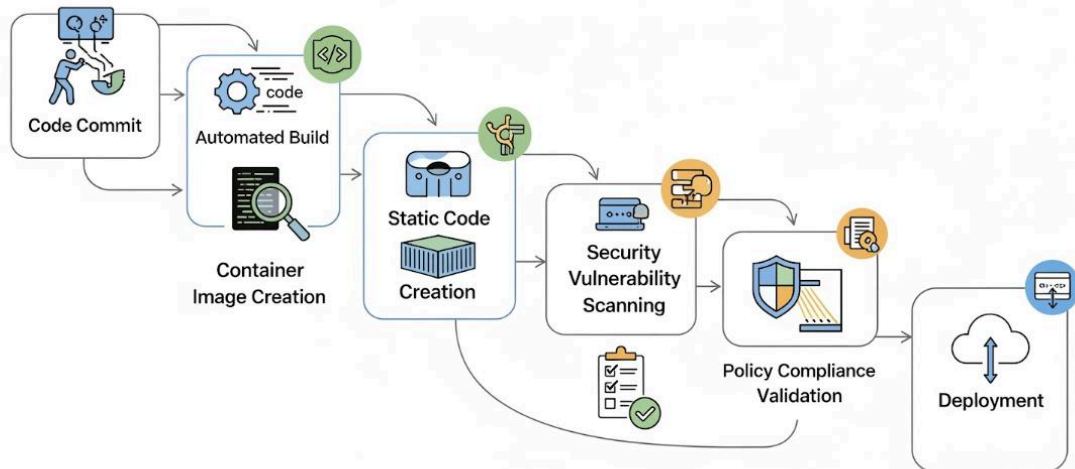
11. DevOps / CI/CD Diagrams ★★

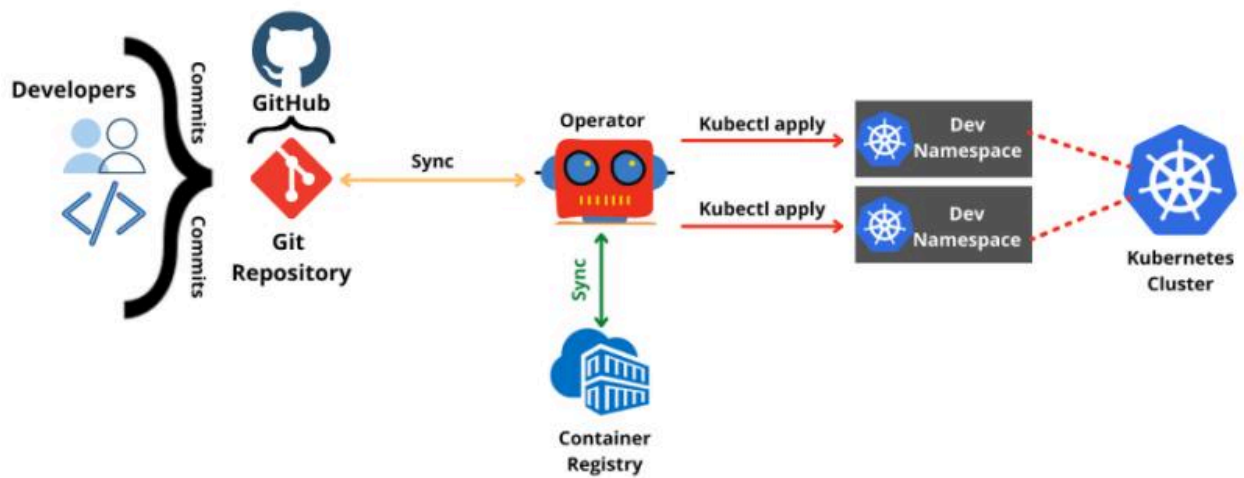
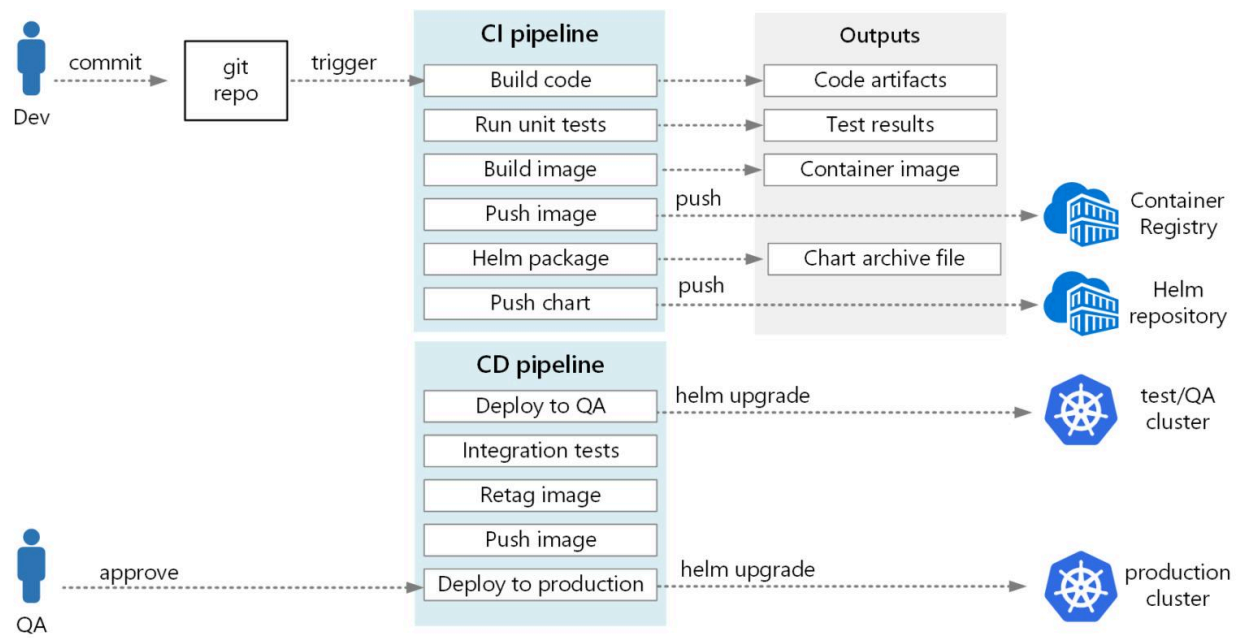


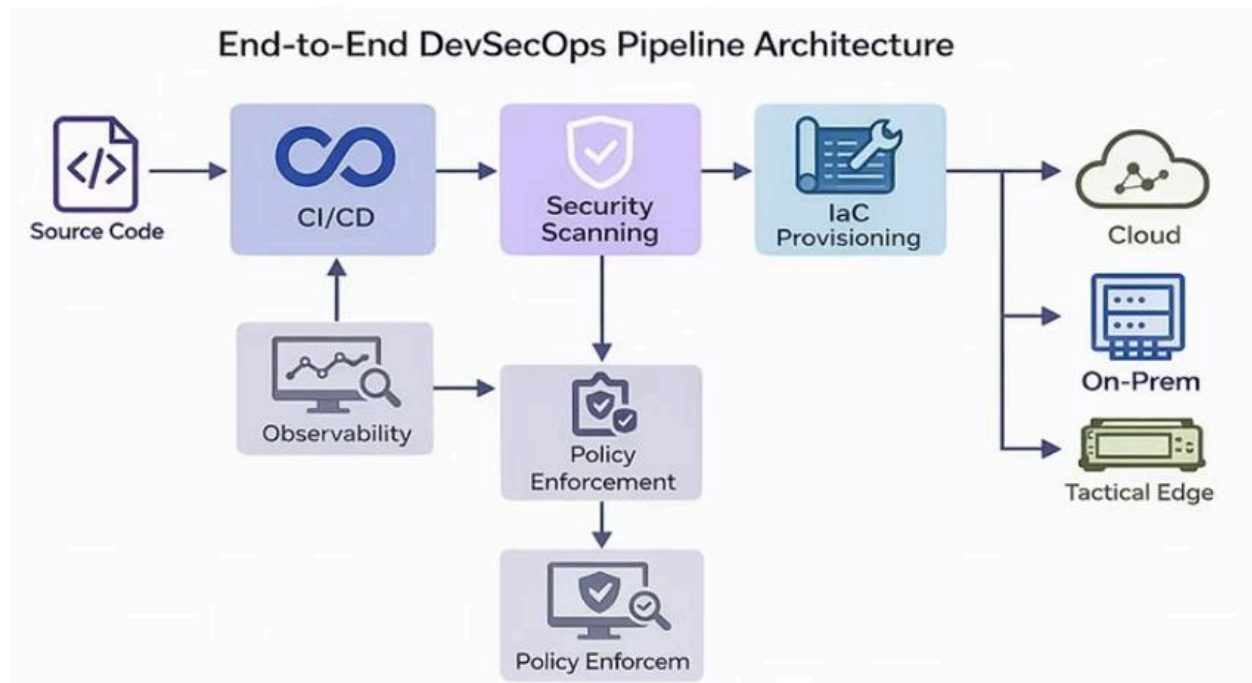
GitOps Architecture



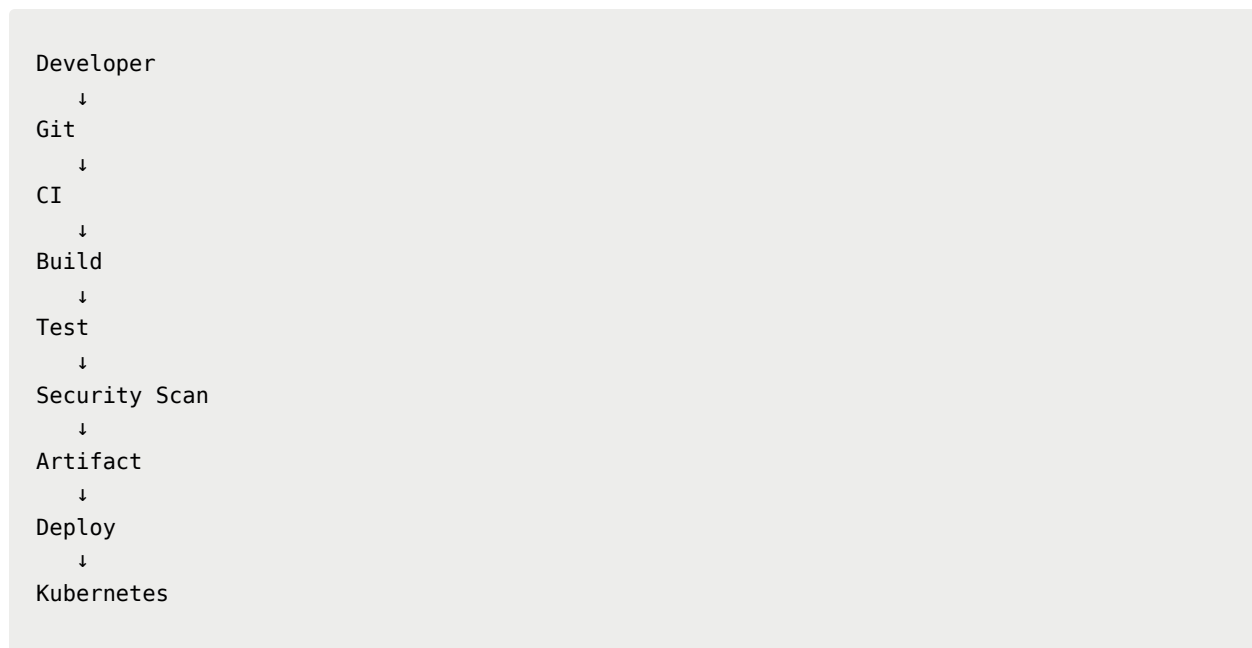
DevSecOps Pipeline







Know how to represent:



Related diagrams:

- **CI/CD Pipeline**
- **Deployment Pipeline**

- **GitOps Architecture**
 - **DevSecOps Pipeline**
 - **Environment Promotion Diagram**
 - **Release Strategy Diagram**
 - **Blue/Green Deployment**
 - **Canary Deployment**
-

12. Observability Diagrams ★★

Very useful for production architecture.

- **Observability Architecture**
- **Logging Architecture**
- **Metrics Architecture**
- **Tracing Architecture**
- **Monitoring Architecture**
- **Alerting Architecture**

Example:

```
Microservices
|
+---- Logs ----> OpenSearch
|
+---- Metrics -> Prometheus
|
+---- Traces --> OpenTelemetry
                    |
                    v
                Observability
```

13. Reliability & Resilience Diagrams ★★

Architects should be able to explain failure behavior visually.

- **Failure Mode Diagram**
- **Resilience Architecture**
- **Fault Tree Analysis (FTA)**
- **Reliability Block Diagram**
- **Disaster Recovery Diagram**
- **Failover Diagram**
- **Circuit Breaker Flow**
- **Retry Flow**
- **Bulkhead Architecture**
- **Active/Active Architecture**
- **Active/Passive Architecture**

Example:

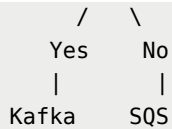
```
Primary Region
  |
  X failure
  |
  v
Secondary Region
```

14. Architecture Decision Diagrams ★★

These aren't traditional "architecture diagrams," but they're extremely useful.

Decision Tree

```
Need asynchronous communication?
  |
  Yes
  |
  Need ordering?
```

Trade-off Matrix

For example:

Option	Cost	Complexity	Scalability
Monolith	Low	Low	Medium
Modular Monolith	Medium	Medium	High
Microservices	High	High	Very High

Other useful artifacts:

- **Architecture Decision Record (ADR)**
- **Decision Matrix**
- **Trade-off Matrix**
- **Option Comparison Diagram**

15. Specialized Architecture Diagrams

You don't need these every day, but a senior architect should recognize them.

Microservices

- Service Landscape
- Service Dependency Diagram
- Service Interaction Diagram
- Microservices Deployment
- Service Mesh Architecture

Serverless

- Serverless Architecture

- Function Dependency Diagram
- Event Trigger Diagram

AI/ML

- ML Architecture
- ML Pipeline
- RAG Architecture
- LLM Application Architecture
- Vector Database Architecture
- AI Agent Architecture

IoT

- IoT Architecture
- Device-to-Cloud Architecture
- Edge Computing Architecture

The diagrams I'd prioritize for a Software Architect

If you're preparing for a **Software Architect / Solution Architect interview**, don't try to master 50+ diagrams equally.

I'd rank them:

Tier 1 — Must know

1. **System Context Diagram**
2. **C4 Container Diagram**
3. **C4 Component Diagram**
4. **Sequence Diagram**
5. **Deployment Diagram**

6. **Cloud Architecture Diagram**
7. **Data Flow Diagram**
8. **ERD**
9. **Domain Model**
10. **State Machine Diagram**
11. **Security/Auth Flow**
12. **Network Diagram**

Tier 2 — Very important

1. **Bounded Context Map**
2. **Event-Driven Architecture Diagram**
3. **CQRS Diagram**
4. **Saga Diagram**
5. **Integration/API Diagram**
6. **CI/CD Diagram**
7. **DR/Failover Diagram**
8. **Observability Architecture**
9. **BPMN / Swimlane**
10. **Data Pipeline / Data Lineage**

Tier 3 — Know when to use

1. Event Storming
2. Event Sourcing
3. Kafka topology
4. Service Mesh
5. Kubernetes architecture
6. Zero Trust

7. Threat Model
 8. Fault Tree
 9. Reliability Block Diagram
 10. Serverless architecture
 11. AI/LLM architecture
 12. IoT architecture
-

A very useful way to remember them

Instead of memorizing diagram names, think about the **question you're trying to answer**:

Question	Diagram
What is our system?	Context
What are its major pieces?	C4 Container
What's inside a service?	Component
How do requests flow?	Sequence
How does data move?	Data Flow
How is data structured?	ERD
How is the domain structured?	Domain Model
How do services communicate?	Integration / Event Flow
Where does it run?	Deployment
How is the network structured?	Network
How does authentication work?	Security/Auth Flow
What happens when something fails?	Resilience / Failover
How is it deployed?	CI/CD
How do we monitor it?	Observability
How are business processes executed?	BPMN / Workflow
Why did we choose this architecture?	ADR / Decision Matrix

The most important skill isn't knowing how to draw all these diagrams. It's knowing which diagram communicates a particular architectural decision most effectively.

For someone with a **.NET + microservices + DDD + event-driven + AWS/Azure** background, I'd especially focus on **C4, sequence, deployment/cloud, DDD bounded-context, event-driven, data-flow, security, and resilience diagrams.**